TECHNICAL REPORT · 初稿 · 2026-10-26

MotionWeaver：面向游戏角色动画的离散姿态 Token 生成与数据管线

MotionWeaver: Discrete Pose-Token Generation and Data Pipeline for Game Character Animation

赵梓宁

独立研究者

zzn1982638980@gmail.com

本报告中的所有数值均读取自仓库内已提交的证据文件（`docs/experiments/assets/`），每张图与每张表都可由 `paper/generate_figures.py` 重新生成。

摘要

本报告记录 MotionWeaver 的技术现状：一个把离散姿态 Token用于游戏角色全身姿态生成的实验系统，以及支撑它的数据管线。系统在 NVIDIA GR00T-WholeBodyControl / MotionBricks 的架构基础上裁剪，以 VQ-VAE 把逐帧骨架姿态压缩为离散码，再以 Pose Token 主干在给定前 4 帧历史姿态与一条独立的 Actor 世界轨迹的条件下预测未来姿态 Token，最后经解码器还原为世界空间骨架。数据侧，我们完成了 1,722 段 UE 动画（79 骨、30 FPS）与 71,132 条 SOMA 通用动作（78 节点、120.0048 FPS）的接入与处理，并把处理吞吐提高到约 1.69–1.80 倍，且输出逐数组等价。在此基础上建成训练视图 72,933 条（覆盖登记总量 73,101 条的 99.77%）、训练窗口 116.6 万个，四个场景 A/B/C/D 分别为 28,233 / 14,820 / 2,743 / 27,137 条。训练侧在单卡 A800 上从零训练 20,000 步，固定验证口径下的掩码姿态 Token 交叉熵由 2,000 步的 1.862 nats 降至 1.454 nats（−21.9%）。训练工程侧，我们让训练端直接读取四键 NPZ 并在 worker 内整批计算特征，单条特征成本由 125.6 ms 降至 0.4 ms（约 16×），并把旋转从 float64 的 3×3 矩阵 改为 float32 四元数，使单条读取由 96.8 ms 降至 18.4 ms（5.25×）、体积压缩 2.12 倍；契约生成与全量评测改为多进程后分别约 12 分钟与 16 分钟，并实测出「32 进程反而把吞吐从 6,350 降到 1,400 条/分钟」这一并发上限（§6）。在此基础上，分词器首次在 full-v1 全库 fps120 组上训满 3 个 epoch（42,791 步），并以 A/B 对照确定码本初始化应保持随机初始化：总损失中位数 0.4026 对 0.5165（§8.7）。本报告还记录了一次被口径掩盖的失败。我们试图让训练目标直接对齐验收指标，把米制关节位置误差写进损失；在 17 组同步数对照下该损失的效应只有 +0.10 cm，小于同配置换种子造成的 0.56 cm 抖动（符号检验 $p = 0.75$），因此判定无效。更有价值的发现来自排查过程：训练与评测此前喂给模型的不是同一个归一化分布，这使得 30,000 步之后的关节误差断崖式恶化（108 → 310 cm），并污染了那一段时期的全部对照结论。把口径对齐后，同一评测口径下的关节 RMSE 由 94.54 cm 降至 83.85 cm（−11.3%）——本项目至今最大的一次单点收益并非来自任何损失项。失效的直接原因也被定位：码本在 20,000 步后塌缩至 `perplexity = 1.000`，每批所有 token 量化到同一码字，分词器退化为自编码器，码字里没有可供优化的信息（§9）。但本报告同样记录失败：在 3 条同轨迹对照片段中，Walk 的世界关节 RMSE 由基线 6.72 cm 改善至 6.03 cm，而 Run（7.36 → 12.00 cm）与 Crouch（5.87 → 8.25 cm）明显退化；接触期脚滑整体仍显著高于来源动作；换用另录的 CMC 轨迹后，90° 转向只改变 Actor 局部关节姿态 0.50 cm，且蹲伏状态未被模型响应。我们据此明确当前系统的能力边界：姿态 Token 预测的学习信号确实在改善，但从「Token 交叉熵下降」到「玩家可操控时姿态与位移一致」之间仍缺一条已验收的链路。

Abstract. This report documents MotionWeaver, an experimental system that generates full-body character poses through discrete pose tokens, together with the data pipeline behind it. Built on the NVIDIA GR00T-WholeBodyControl / MotionBricks architecture, a frozen VQ-VAE quantizes per-frame skeletons into codebook indices; a pose-token backbone then predicts future tokens from four history frames and an independent actor world trajectory. On the data side we ingest 1,722 Unreal Engine clips (79 bones, 30 FPS) and 71,132 SOMA clips (78 joints, 120.0048 FPS), and improve processing throughput by 1.69–1.80x with array-level equivalence. A 20,000-step training run on a single A800 reduces masked pose-token cross-entropy on a frozen validation split from 1.862 to 1.454 nats (-21.9%). We further document a failure hidden by a protocol mismatch. Adding a metric joint-position term to the training loss produced an effect of +0.10 cm over 17 matched-budget comparisons—smaller than the 0.56 cm seed-to-seed spread (sign test

p=0.75)—so the term is judged ineffective. The diagnostic that mattered was upstream: training and evaluation had been consuming *different normalization distributions*, which drove joint error from 108 to 310 cm past 30k steps and invalidated every conclusion drawn from that regime. Aligning the protocols lowers joint RMSE from 94.54 to 83.85 cm (-11.3%) under one fixed protocol—the largest single gain in this project, and it came from no loss term at all. The proximate cause of the loss term's ineffectiveness is also identified: the codebook collapses to perplexity 1.000 after 20k steps, quantizing every token in a batch to the same codeword, so the tokenizer degenerates into an autoencoder and there is no codeword information left to optimize (Sec. 9). We also report failures: on three same-trajectory clips, walk improves (6.72 to 6.03 cm joint RMSE) while run (7.36 to 12.00 cm) and crouch (5.87 to 8.25 cm) degrade; contact-phase foot sliding remains far above the source motion; and replacing the trajectory with a recorded CMC capture changes local joint poses by only 0.50 cm under a 90 degree turn, with crouch state unobserved. We therefore state the capability boundary explicitly rather than inferring playable quality from a falling token loss.

形式上，本报告把它定义为一个以离散姿态码为目标的掩码条件预测问题（§3）：**给定 h 帧历史姿态与一条外部** Actor **轨迹，最大化未来姿态码的对数似然**（式 3.3）。**损失函数、评测指标与硬约束投影都写成可复核的显式定义，使每个数字都能追溯到它对应的数学对象**（§5、§7）。

关键词：**动作生成** · **离散姿态** Token · VQ-VAE · **角色动画** · Unreal Engine · **数据管线** · **评测口径** · **码本塌缩**

---

1 引言

游戏角色的全身动画长期由动画状态机、混合空间与 Motion Matching **等检索式方法主导。这类方法在确定性与可调试性上有明显优势，但其表现力被资产库的覆盖范围所限制：玩家做出资产库没有的组合动作时，系统只能退回到不匹配的片段或生硬的混合。近年来，学习式动作生成在动作补间、文本驱动动作以及物理角色控制上取得进展，使「由条件实时合成姿态」成为一个现实的工程选项。**

MotionWeaver **的目标是一个具体的工程问题：在** Unreal Engine **中，让模型根据玩家控制器的运动状态与历史姿态，持续生成与该位移一致的全身姿态。我们把世界位移**（Root）**的权威留给引擎的运动组件**（CMC / Mover），**模型只负责生成** Actor **相对的身体姿态。这个分工不是架构偏好，而是把「碰撞、网络权威、可回滚的位移」留在引擎侧，把「姿态合理性」交给模型。**

技术路线上我们沿用 NVIDIA GR00T-WholeBodyControl **仓库中** MotionBricks **的做法：先把动作离散化为姿态** Token，**再在** Token **空间做序列预测。离散化的好处是模型只需预测有限码本上的索引，误差不会像逐关节回归那样直接累积到厘米级的关节位置上；代价是量化误差与解码器条件的耦合，这一点在** §8.6 **的** oracle **诊断里被量化了。**

本报告的三点贡献是：

- **一条可追溯的双来源数据管线。**UE **导出与** SOMA **通用动作在统一的** 78 **节点目标骨架上汇合，每条** clip **带内容哈希、帧率分组与显式边界声明**（§3）。
- **一组可复现的固定口径评测。冻结验证集、冻结划分、冻结随机种子，同时报告** Token **级指标、世界姿态指标、接触指标与控制器条件探针**（§7、§8）。
- **一份如实的能力边界陈述。我们把退化案例、缺失条件以及「哪些数字不能当成在线能力」逐条写进报告与附录**（§9）。

> **阅读方式 —— 本报告刻意把「训练指标改善」与「产品可用」分成两件事陈述。**§8.1–§8.2 **是学习信号，**§8.3–§8.5 **是离线质量，**§8.6 **是上限诊断，**§9 **是缺陷清单。任何一节单独拿出来都不足以支撑「模型可用」的结论。**

2 相关工作

2.1 离散动作表示与生成

VQ-VAE 把连续信号映射到有限码本上的离散索引，是离散生成式建模的基础构件 [3]。 在图像领域，VQGAN 展示了「离散 Token + 自回归先验」的有效性 [4]； 在人体动作领域，T2M-GPT [7] 与 MoMask [8] 沿同一思路把 HumanML3D [5] 的动作序列压缩为 Token 再做生成； 扩散模型路线则由 MDM [6] 等代表。MotionWeaver 属于前者， 但把 Token 的预测任务限定为「短历史 + 已知外部轨迹 → 未来姿态」， 而不是「文本 → 整段动作」。

2.2 目标驱动的角色动画

面向目标（trajectory / keyframe / contact）的动画方法大致分两支。 程序化一侧以 Motion Matching 为代表，把当前姿态与目标特征放到同一空间里做最近邻检索 [9]，并用 Learned Motion Matching 把检索网络化 [10]。 学习一侧用相位、VAE 或扩散模型生成中间姿态：Local Motion Phases 用相位变量组织多接触动作 [11]，Motion VAE 控制器把状态与潜变量一起驱动角色 [12]， Robust Motion In-betweening 用带噪插值策略学习关键帧之间的补间 [13]， 扩散式方法进一步放宽了关键帧的约束形式 [14]。

MotionWeaver 的位置介于两支之间：它借用学习式生成来表达资产库之外的姿态， 但把确定性约束（步幅、朝向、脚部贴地）交给 Warping 与 IK 这类程序化模块（§5.4）， 并用阈值化的回退路径保证失败可控。

2.3 与上游项目的关系

本仓库基于 [1, 2] 的架构与参考实现，保留了 VQ-VAE 分词器、 姿态主干与 G1 参考演示；面向 UE 的裁剪包括：79 骨（后为 78 节点 SOMA 骨架）的姿态表示、 独立 Actor Root sidecar、玩家状态条件接口，以及 Windows 本地推理与导出工具。 上游权重与本仓库训练权重严格区分，报告中的任何指标都来自本仓库自己的检查点。

3 问题形式化

本节把「由玩家控制状态生成全身姿态」写成显式优化问题。公式里出现的每个符号都在下表定义， 后续所有损失、指标与约束只用这些符号。这样做不是因为偏好形式化，而是为了修掉我们在 §9.3 记录的三次误判——它们的共同点是同一个符号在不同章节指代了不同对象。

	时间与索引
$T,\ h$	窗口帧数与历史帧数。本系统 $T=24$，$h=4$（30 FPS 下 h 恰为 133 ms）。
$t,\ \ell$	帧索引 $t\in\{1,\dots,T\}$；Token 索引 $\ell\in\{1,\dots,L\}$。
	骨架与位姿
J	关节数。UE 侧 $J=79$（骨），SOMA 侧 $J=78$（节点）。
$\mathbf{p}_t^j\in\mathbb{R}^3$	第 t 帧第 j 关节的世界位置，米制。
$\mathbf{R}_t^j\in SO(3)$	第 t 帧第 j 关节的世界旋转（单位四元数或 3×3 正交阵）。
$\mathbf{q}_t\in\mathbb{R}^7$	第 t 帧 Actor 世界轨迹位姿 (x,y,z,q_x,q_y,q_z,q_w)，Y 轴向上。
$\mathbf{b}_t^j$	Actor 相对的身体位姿，$\mathbf{b}_t^j=\mathbf{q}_t^{-1}\odot(\mathbf{p}_t^j,\mathbf{R}_t^j)$。
	离散码本
K	码本大小（Token 词表规模）。
$\mathcal{C}=\{\mathbf{e}_k\}_{k=1}^K$	码本，$\mathbf{e}_k\in\mathbb{R}^d$。
$Z=\{z_t\}_{t=1}^T$	姿态 Token 序列，$z_t\in\{1,\dots,K\}$。
	损失与阈值
$\mathcal{M}$	被遮住的位置集合，$\lvert\mathcal{M}\rvert=\lceil rT\rceil$，本系统 $r=0.8$。
ϵ	接触期容许脚速，来源动作实测 0.03–0.07 m/s。
λ	硬约束投影中的平滑权重。

3.1 姿态的离散化

编码器 E_ϕ 把一段身体位姿压成 L 个潜向量，解码器 D_ψ 在外部条件下还原：

$$\mathbf{u}_{1:L} = E_\phi\big(\mathbf{b}_{1:T}\big), \qquad \hat{\mathbf{b}}_{1:T} = D_\psi\big(\hat{\mathbf{u}}_{1:L},\, \mathbf{c}^{\text{ext}}\big). \tag{3.1}$$

量化器把每个 $\mathbf{u}_\ell$ 吸附到码本中最近的一项：

$$z_\ell = \arg\min_{k\in\{1,\dots,K\}} \big\|\mathbf{u}_\ell - \mathbf{e}_k\big\|_2, \qquad \hat{\mathbf{u}}_\ell = \mathbf{e}_{z_\ell}. \tag{3.2}$$

码本由多头 EMA 更新而非梯度优化。因此主干训练时量化步是冻结的： 主干看到的 z_ℓ 是常量标签，学习问题退化为干净的多类分类（§5.2）。 这一点很关键——它使「Token 交叉熵」可以被单独报告，而不与重建误差混为一谈。

3.2 条件预测任务

给定 h 帧历史身体姿态与未来 T 帧外部 Actor 轨迹，预测未来姿态码：

$$p_\theta\big(Z \mid \mathbf{b}_{1:h}, \mathbf{q}_{h+1:h+T}\big) = \prod_{t=1}^{T} p_\theta\big(z_t \mid \mathbf{b}_{1:h},\, \mathbf{q}_{h+1:h+T},\, z_{<t}\big). \tag{3.3}$$

条件里不含 $\mathbf{b}_{h+1:h+T}$：未来真实姿态只作为监督目标。 另一条更隐蔽的约束是 $\mathbf{q}_{h+1:h+T}$ 在推理期不可得——游戏里它只能来自 CMC/Mover 或规划器，不可能等于真值。因此凡是使用 $\mathbf{q}^{\text{true}}$ 的数字都只在 「配对教师轨迹」口径下成立，这一点在 §5.2 与 §9.2 各强调一次。

3.3 训练目标

采用掩码建模目标：随机遮住比例 r 的姿态码，只对被遮住的位置计损失，

$$\mathcal{L}_{\text{token}}(\theta) = -\frac{1}{|\mathcal{M}|} \sum_{t\in\mathcal{M}} \log p_\theta\big(z_t \mid \mathbf{b}_{1:h},\, \mathbf{q}_{h+1:h+T},\, Z_{\setminus\mathcal{M}}\big). \tag{3.4}$$

单位是 nats（自然对数），这也是全文所有「Token CE」的口径。 VQ-VAE 一侧的损失（本报告中被冻结，不参与主干训练）为

$$\mathcal{L}_{\text{vq}} = \big\|\mathbf{b}_{1:T} - \hat{\mathbf{b}}_{1:T}\big\|_2^2 + \big\|\text{sg}[\mathbf{u}] - \hat{\mathbf{u}}\big\|_2^2 + \beta\big\|\mathbf{u} - \text{sg}[\hat{\mathbf{u}}]\big\|_2^2, \tag{3.5}$$

三项依次是重建、码本与承诺，$\text{sg}[\cdot]$ 为停止梯度算子。 它们的分工是：重建管解码质量，码本管码本更新，承诺管编码器不要频繁跳到别的码。

3.4 接触约束与它的去向

若记脚部接触指示为 $c_t^{\text{foot}} \in \{0,1\}$，接触期的理想约束是

$$\Big\|\tfrac{\mathrm{d}}{\mathrm{d}t}\mathbf{p}_t^{\text{foot}}\Big\|_2 \le \epsilon \qquad \text{当} \quad c_t^{\text{foot}} = 1. \tag{3.6}$$

式 (3.6) 不进入 $\mathcal{L}_{\text{token}}$。 我们把脚部约束交给输出端的 Foot Lock 与 Foot IK（§5.4）， 模型只负责姿态合理性。§8.3 报告的 0.78–0.88 m/s 就是式 (3.6) 未被满足的量化形式。

4 数据管线

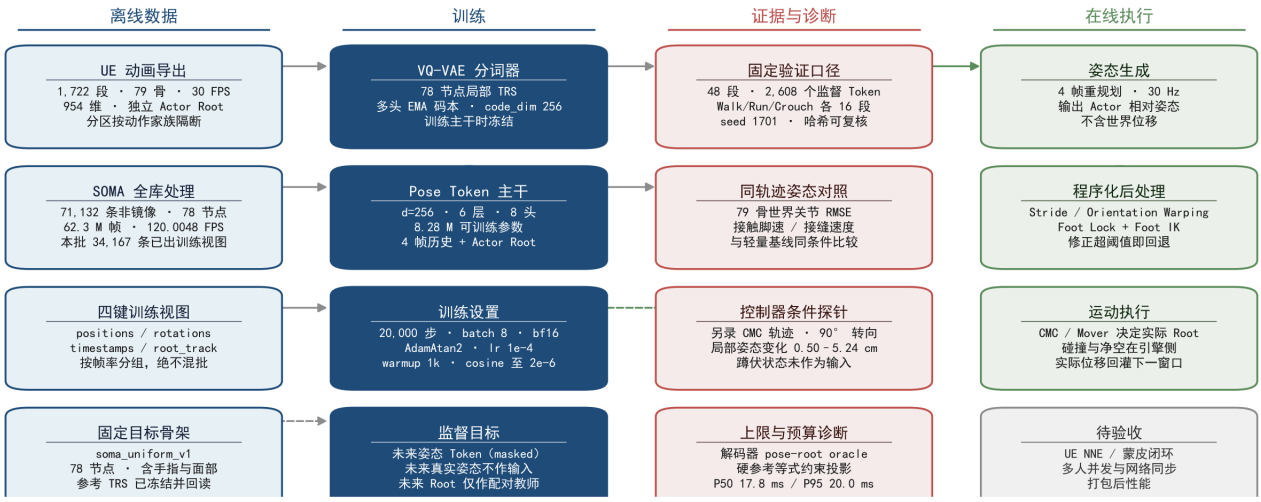


图 1：系统总览。左起四条泳道依次是离线数据、训练、证据与诊断、在线执行。 实线表示已有产物与权重的实际路径；虚线表示已定义但尚未验收的接口（例如 UE NNE 闭环、 模型自驱 Root）。图中所有数值取自仓库内已提交的证据文件， 完整生成脚本见 `paper/generate_figures.py`。

4.1 两个来源，一套目标骨架

训练数据来自两条互不相同的管道。第一条是项目自有 UE 动画：从 `AnimSequence` 导出骨架与逐帧姿态，得到 1,722 段 79 骨、30 FPS、954 维的动作， 划分 train 1,355 / validation 189 / test 178，三个分区的动作家族（asset family）两两交集为 0。 这条管道的关键产物是一份独立的 Actor Root sidecar：每段附一条 `[frames, 7]` 的米制 Y-up 世界轨迹（xyz + xyzw 四元数），使「身体姿态」与「世界位移」在数据层就分离开。

第二条是SOMA / BONES-SEED 通用动作库：71,132 条非镜像记录、62,301,201 帧， 最短 69 帧、最长 21,617 帧、中位 685 帧，来源声明帧率约 120.0048 FPS。 这套数据使用 78 节点的 `soma_uniform_v1` 固定骨架，含手指与面部关节。

两条来源的骨架不同（79 骨 vs 78 节点），但都汇入同一个目标骨架定义。 处理时遵循一条硬约束：原始数据永不改写。派生表示一律写入新批次目录， 并以内容哈希绑定；回读校验的门槛是 Root/身体重合成误差不超过 `1e-9 m`， 实测四轮试点的最大误差为 `8.88e-15 m`。

4.2 全链路：我们改了什么

本仓库不是纯粹的上游复刻。为免读者把上游能力与本项目改动混为一谈，下表逐层列出 本项目实际修改或新增的模块，以及它在本报告中的证据位置。未列出的部分沿用上游实现。

表 1：全链路改动清单。"层"按数据流动方向排列；"证据"指本报告中给出结果的章节，或仓库内对应的实现文件。

层	本项目改动	关键文件	证据
导出	UE 编辑器插件新增逐片段 SHA-1 内容身份、引擎/插件版本与导出时间戳；记录 `clip_identity` 而不破坏既有 `clips` 契约	`AILocomotionDatasetEditorModule.cpp`	§4.1
校验	导入前强制核验导出清单的内容身份，不匹配即中止，不留半成品目录	`prepare_unreal_dataset.py`	§4.1
状态诚实性	重定向未解决的条目从 `normalized` 改为 `needs_skeleton_review`；审核侧支持批次显式声明纳入状态	`retarget_uefn_to_soma.py` `native_review_store.py`	§4.3
训练视图	从 Hips 合成可逆 Root 代理；四键 NPZ；按场景 × 帧率分组，绝不混批	`soma_train_dataset.py`	§4.2、§4.4、§4.5
批处理	复用世界坐标、直接算文件字节 SHA、每进程缓存骨架与目录、1 MiB 写缓冲	`process_soma_dataset.py`	§4.7
契约与评测	把训练视图封装成既有评测器可读的条件契约；一条命令跑完契约→评测→汇总	`build_soma_contract.py` `run_soma_eval.py`	§6.1
模型	Actor Root 条件 profile（4 帧历史）；Pose Token 主干 8.28 M 可训练参数，VQ 冻结	`motionweaver_pose_model.py`	§5.1、§5.2
训练数据接入	直接读四键 NPZ（不预展开 918 GB 稠密数组）；特征在 worker 内整批计算；旋转改存 float32 四元数	`soma_realtime_dataset.py` `compress_training_views.py`	§6.1–§6.3

并行与参数	契约与评测按窗口分片改多进程；新增 `--soma_views` / `--num_workers` 等运行时参数；训练污染主动核查	`build_soma_contract_parallel.py`
		`evaluate_conditioned_baselines_parallel.py`
		`audit_training_contamination.py`
			§6.4–§6.6

训练	从零 20k 步；固定验证口径；检查点哈希双向核对	`pretrain/train_unreal.py`	§8.1、§8.2
推理	给定 Actor Root 的本地推理入口；同轨与控制器轨迹双口径评估	`inference/runtime/motionweaver.py`	§8.3、§8.5、§8.7
约束后处理	硬参考姿态的等式约束投影（最小范数 + 二阶差分平滑）	`hard_pose_constraints.py`	§5.5
回归	索引重建、片段身份、未解决状态三组测试	`tests/test_*.py`	附录 A

未列入的层（VQ-VAE 分词器结构、Pose Backbone 主干、评测器内部实现）沿用上游 MotionBricks 与本项目早期版本，其来源与许可见仓库 `引用与许可说明.md`。表中"改动"均已有对应提交或工作树变更，不是规划项。

4.3 Root 的离线表示：代理而非真值

SOMA 的 `Root` 节点在多数片段中是静止的，真实运动合并在 `Hips` 上。因此训练用的 `root_track` 是一条从 Hips 派生的代理轨迹：水平位置与朝向取自 Hips，垂直高度全部保留在身体姿态里。该代理可逆（回读可完全恢复录制位置），但它不是场景中观测到的 Actor Root，输出中显式标注 `root_status = offline_proxy_not_observed_actor_root`。

形式化地，记 Hips 世界位置为 $\mathbf{h}_t = (h_x, h_y, h_z)$，则代理 Root 与身体位姿定义为

$$\mathbf{q}_t^{\text{proxy}} = \big(h_x,\, 0,\, h_z,\, \psi_t\big), \qquad \tilde{\mathbf{b}}_t^{\,j} = R_y(\psi_t)^{-1}\big(\mathbf{p}_t^{\,j} - (h_x, 0, h_z)\big), \tag{4.1}$$

其中 ψ_t 为左右髋形成的前向偏航角。注意竖直分量 h_y 留在身体位姿里，因此回读可精确恢复录制位置：

$$\mathbf{p}_t^{\,j} = R_y(\psi_t)\,\tilde{\mathbf{b}}_t^{\,j} + (h_x, 0, h_z), \qquad \max_{j,t} \big\| \hat{\mathbf{p}}_t^{\,j} - \mathbf{p}_t^{\,j} \big\|_\infty \le 10^{-9}\ \text{m}. \tag{4.2}$$

实测四轮试点的最大重合成误差为 8.88×10^{-15} m，比门槛低五个数量级。但必须先说清：这是离线代理，不是场景观测到的 Actor Root。

这个区分不是学术洁癖。2026-09-28 **的实验中，早期把** `motion_*.npy` **里的** 5 **维** root（**实为** Actor **相对的骨盆特征**）**误当作** CMC Root **使用，会直接污染「模型能否跟随控制器轨迹」的结论；发现问题后改为逐片段重建** sidecar，**并用「复跑所得** 1,722 **个文件与上传内容逐字节一致」来确认修复。**

4.4 接触候选：区分「未知」与「非接触」

脚部接触是逐帧四值状态： `-1 unknown` 、 `0 noncontact_candidate` 、 `1 contact_candidate` **，另有手部始终为未知。规则是几何与运动学的联合判据：脚使用踝/趾代理点在完整录制空间的速度（不是骨盆相对速度），并要求持续时间、进出速度与高度变化同时满足；每只脚可保留多个稳定代理高度，避免把高台上的脚因远离全局最低点而一律判为腾空。**

必须强调的是： `contact_known_mask` **只表示「有候选值」，不表示人工确认或传感器真值。本报告中的所有「接触脚速」指标都建立在这层启发式标签上，用来做相对比较，不作为绝对物理量。**

4.5 双帧率：用 Token 时间粒度对齐，而不是用帧数

UE **来源是** 30 FPS，SOMA **主库是** 120.0048 FPS。**直接把两者混进同一个** batch **会破坏** Token **的时间语义：同一个** `down_t` **在** 120 FPS **下是** 133 ms，**在** 30 FPS **下却变成** 533 ms。**我们的做法是让每个** Token **覆盖的时间一致。设基准帧率** $f_0 = 120.0048$、**编码器降采样层数** δ（**即** `down_t`），**则单个** Token **覆盖**

$$\tau(f, \delta) = \frac{2^{\delta}}{f} \quad \text{秒}, \tag{4.3}$$

要让两种帧率语义等价，只需 $\tau(f_1, \delta_1) = \tau(f_0, \delta_0)$，**即**

$$\delta(f) = \delta_0 - \log_2 \frac{f_0}{f} = \delta_0 - \operatorname{round}\!\left(\log_2 \tfrac{f_0}{f}\right). \tag{4.4}$$

代入 $f_0 = 120.0048$、$f = 30$ **得** $\delta(f) = \delta_0 - 2$。**具体取值：**

```
down_t(fps) = down_t(base) - round(log2(base_fps / fps))      base_fps = 120.0048

场景 A（常规移动，窗口 0.4 s）
    120fps + down_t=4  ->  16 帧/Token  =  133.33 ms
     30fps + down_t=2  ->   4 帧/Token  =  133.33 ms

场景 B/C（大姿态转换 / 跑酷，0.6 s）
    120fps + down_t=3  ->   8 帧/Token  =   66.67 ms
     30fps + down_t=1  ->   2 帧/Token  =   66.67 ms
```

两侧的时间误差约 0.004%，**仅来自** `120.0048` **与** `30` **的舍入差。但注意：** `down_t` **在本架构中就是编码器的** stride-2 **卷积层数，层数不同意味着权重不可共享，因此每个（场景 × 帧率）组合需要独立训练分词器。这一条我们在早期方案里判断错了，后来按源码** `encdec.py:26-32` **纠正。**

4.6 训练视图与当前构建进度

训练视图是一个派生层：每条 clip **输出四键** NPZ（ `positions / rotations / timestamps / root_track` ），**带** `fps` **与** `fps_group` **，按组写入不同子目录，绝不混批。构建过程只读源数据，失败时不写** `dataset.json`（**发布屏障**）。

表 2：训练视图的最终统计量。数据由本次完成的构建的 `dataset.json` 直接聚合得到，覆盖登记总量 73,101 条中的 72,933 条。

统计项	数值	说明
已构建 clip 数	72,933	**占登记总量 73,101 的 99.77%**
累计源帧数	62,525,689	**单帧即一个 78 节点的局部 TRS 样本**
片段长度 中位 / p10 / p90	673 / 333 / 1,501 **帧**	**最短 19 帧，最长 21,617 帧**
fps120 **组**	71,127 **条** · 62,300,282 **帧**	**折合 144.2 小时**
fps30 **组**	1,806 **条** · 225,407 **帧**	**折合 2.1 小时**（UE **重定向来源**）
场景 A（常规移动）	28,233 **条**	walk / run / jog / idle / turn / strafe
场景 B（大姿态转换）	14,820 **条**	jump / land / crouch / kneel / lying
场景 C（跑酷攀爬）	2,743 **条**	climb / vault / mantle
场景 D（开放交互）	27,137 **条**	**手势 / 舞蹈 / 物体操作 / 坐姿**
训练窗口总数	1,166,446	fps120 1,151,751 + fps30 14,695
被拒绝条目	168	**占 0.23%，全部为帧数不足的引擎姿势资产**

注：被拒条目经人工复核，167 条为 7–71 帧的引擎资产（`PoseAssets`、`AimOffset`、`CurveContainer`、`Ragdoll` 短片段），在 Unreal 中作为叠加姿势而非完整可播放动作，不构成训练样本；另有 1 条源数据含非有限值。对全量 72,933 条做路径模式扫描，`PoseAssets` / `AimOffset` / `CurveContainer` / `Montage` 命中数均为 0，仅 30 条 61–249 帧的 `Ragdoll` 挣扎动画被判定为真实时间序列动作而保留，占 0.041%。

4.7 处理吞吐：把 I/O 等待当成一等问题

全库处理最初的瓶颈不在计算而在文件系统。对一条 3,769 帧动作做 profile，单条耗时 19.870 s，其中文件打开累计 6.912 s、目录创建 5.015 s、`stat` 2.084 s、替换 1.851 s，而真正的压缩只占约 0.320 s。优化保持数学过程不变，只做四件事：复用已计算的世界坐标、回读时直接算文件字节 SHA、每进程缓存固定骨架与已建目录、把写缓冲从默认值提到 1 MiB。没有任何回读校验被删减，精度没有降低，也没有降采样。

表 3：同一 8 条样本、顺序运行的端到端计时（含工作进程启动与落盘回读）。每轮输出到独立 run ID；OS 缓存未清空，因此只作同条件对照，不能外推为全库完成时间。

版本	并发	端到端秒数	比较
原版	1	22.524	**并发探测**
原版	4	14.546	**并发探测**
原版	8	11.364	**对照第 1 轮**
优化	4	8.577	**较同并发原版 1.70× 吞吐**
优化	8	6.718	**较第 1 轮 1.69×，耗时** −40.9%
原版	8	11.938	**对照第 2 轮**
优化	8	6.643	**较第 2 轮 1.80×，耗时** −44.4%

等价性：8 条基线分别与优化 4 进程、8 进程及复测版本比较，共 24 次配对；`raw` 与 `processed` 的 SHA-256 完全相同，9 个 NPZ 数组逐元素完全相同。加速比只对本样本集成立，全库平均加速未验证。

5 模型

5.1 两段式：分词器 + 姿态主干

系统由两个可分离的网络组成。左侧是 VQ-VAE 分词器：编码器把一段窗口内的 逐帧局部 TRS 压到潜空间，再经多头 EMA 码本量化成离散索引；解码器把索引还原为姿态特征。 右侧是 Pose Token 主干：一个 6 层、8 头、宽度 256 的 Transformer， 在给定条件下预测未来窗口的 Token 序列。训练姿态主干时，分词器冻结， 因此主干的训练目标是一个干净的「预测哪些码」的分类问题。

表 4：两次训练的关键配置。两轮使用同一 VQ 检查点、同一随机种子， 但调度器总步数不同（2,000 vs 20,000），因此不能当作「只改步数」的消融。

项目	2k 链式短跑	20k 正式训练
硬件	单卡 A800-SXM4-80GB	单卡 A800-SXM4-80GB
精度 / 批量	bf16-mixed / 8	bf16-mixed / 8
随机种子	42	42
优化器	AdamAtan2，lr 1e-4，wd 0	AdamAtan2，lr 1e-4，wd 0
调度	warmup 200 到 cosine，总计 2,000 步	warmup 1,000 到 cosine 至 2e-6，总计 20,000 步
可训练参数	8,278,544（Pose Backbone；VQ-VAE 全冻结）	
条件 profile	`actor_root_history4_compact_v1`：仅 4 帧历史姿态	
监督目标	未来姿态 Token（masked ratio 0.8），未来真实姿态不作输入	
Token 预算	6–16 Token / 窗口；1 Token = 4 帧（down_t = 2）	
有效样本窗口	约 15,900	约 159,415（约 117 个 epoch）

5.2 条件与监督的边界

一个训练样本包含三类信息，其可见性被严格区分：

- 可见条件：前 4 帧真实姿态（作为历史），以及由外部提供的未来 Actor 轨迹。
- 不可见条件：未来真实姿态。它只以 Token 目标的形式出现在损失里。
- 训练期可见、推理期不可得：来源动作的未来 Actor Root。它作为「配对轨迹教师」参与训练， 但在真实游戏里，未来的轨迹只能由 CMC/Mover 或规划器给出，不可能等于真值。

第三点是本报告所有对照实验的解读前提。凡是标注「配对教师轨迹」的结果， 都只说明模型在该轨迹下能生成什么，不说明它在玩家控制下能生成什么。 为此我们另设了一组使用「另录 CMC 轨迹」的探针（§8.5）。

5.3 为什么是 4 帧历史

历史长度决定了推理延迟的下界与信息的充分性。本项目早期的轻量基线 `ReferenceGuidedPose`（约 310 万参数，GRU 历史编码加 3 层 Transformer）使用 24 帧历史； MotionWeaver 的 `actor_root_history4_compact_v1` 只给 4 帧。 这是一个刻意的取舍：4 帧 @30 FPS 是 133 ms，恰好等于一个重规划步长， 使「上一窗口的输出立即成为下一窗口的历史」成为可能。 代价是模型获得的动力学信息更少，这可能是 §8.3 中 Run 退化的原因之一， 但那只是假设：本报告没有做「只改历史长度」的受控实验来证明它。

5.4 模型与程序化的分工

我们不要求模型学会「脚必须贴地」。分工如下：

表 5：职责划分。最后一列的阈值是初始值，需在实测后调整；这些阈值尚未进入任何已验收的闭环，属于设计约定而非实测结论。

层	负责方	理由
世界位移（Root）	CMC / Mover（默认）；Model Root 为可选模式	碰撞与网络权威必须在引擎侧
姿态生成	模型	模型的唯一核心职责
步幅 / 朝向匹配	Stride / Orientation Warping	确定性几何，模型做不准
脚部接触固定	Foot Lock + Foot IK	需要世界空间约束
大姿态转换	模型生成 + 程序化相位门控	显式告知「正在蹲到 40%」比让模型自己发现更可靠
手部环境接触	不判定，由 Gameplay 提供 IK 目标	无场景几何
失败回退	程序化监控	Foot IK 修正超 5 cm，或 Warping 缩放出 [0.8, 1.25]，或朝向偏差超 30°

5.5 硬参考姿态的等式约束投影

除纯生成路径外，系统还支持「给定若干时刻的硬参考姿态」这一条件形式。设生成输出为 $\{\mathbf{o}_t\}_{t=1}^{T}$（局部平移或旋转向量），参考帧集合 $\mathcal{K}$ 上给定目标 $\mathbf{r}_k^\star$。我们求一个修正量 $\{\Delta_t\}$：

$$\min_{\{\Delta_t\}} \sum_{t=1}^{T} \|\Delta_t\|_2^2 + \lambda \sum_{t=2}^{T-1} \left\|\Delta_{t+1} - 2\Delta_t + \Delta_{t-1}\right\|_2^2 \quad \text{s.t.} \quad \mathbf{o}_k + \Delta_k = \mathbf{r}_k^\star,\ \forall k \in \mathcal{K}. \tag{5.1}$$

第一项是最小范数（改动尽量小），第二项是二阶差分（改动尽量平滑，抑制抽搐），等式约束保证参考帧精确到位。这是一个带等式约束的二次规划，用拉格朗日乘子求解，规模为 $|\mathcal{K}|$ 个乘子：

$$\begin{bmatrix} A & C^{\mathsf{T}} \\ C & 0 \end{bmatrix} \begin{bmatrix} \Delta \\ \boldsymbol{\mu} \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{r}^\star - \mathbf{o} \end{bmatrix}, \tag{5.2}$$

其中 $A = I + \lambda D^{\mathsf{T}} D$，$D$ 为二阶差分矩阵，C 为在 $\mathcal{K}$ 上取值的指示矩阵（故 $C\Delta = \mathbf{r}^\star - \mathbf{o}$）。解出后 $\Delta = A^{-1}(I - C^{\mathsf{T}}(CA^{-1}C^{\mathsf{T}})^{-1}C)\,(\mathbf{r}^\star - \mathbf{o})$，与参考相距 8 帧以上的位置 Δ_t 自然为零——这是 $D^{\mathsf{T}}D$ 的局部支撑带来的，不是额外加的阈值。旋转侧对 $SO(3)$ 使用主值旋转向量再以四元数组合回位姿，避免在 π 附近出现分支跳变。

在一个用户手选 6 帧的跑酷翻越片段上（源帧 32/34/44/52/56/59），平均关节误差由 11.322 cm 降至 6.903 cm，加速度误差由 4.008 升至 4.186 cm/帧²。参考帧位置误差为数值零（最大局部旋转误差小于 0.000003°），但这是约束构造的结果，不是模型预测精度。平滑版本明显优于直接覆盖（后者加速度误差恶化到 9.428），却仍不如软输出平滑，因此不能声称已消除抽搐。该路径没有速度等式、接触、碰撞或关节限位保证。

6 训练工程优化

本节记录训练链路上的工程优化。它们不改变模型定义，只解决一件事：让同一份数据在同样的硬件上更快跑完。之所以单独成章，是因为这些改动各自都有一个「看起来更快、其实更慢」的陷阱，而陷阱只有实测能发现。

> **口径声明** —— 本章所有耗时均为本机实测（RTX 4070 Laptop 8 GB、32 逻辑核、Windows、torch 2.14.0+cu130），不是估计值。凡属推算的，文中一律标注「推算」。优化只涉及速度，不涉及训练效果；§9 的所有质量指标仍来自 A800 上那份未改动的 20k 检查点。

6.1 第一个阻塞：训练脚本读不了我们的数据

优化之前先有一个更根本的问题。数据侧产出的是四键 NPZ（`positions / rotations / timestamps / root_track`），而 `UnrealMotionDataset` 要求的是 UE Schema v1/v2：`motion.json` + `arrays.bin`，并强制校验

`schema_version`、`coordinate_system` 与 `stats/mean.npy`、`stats/std.npy`。两者不兼容，训练根本进不去。

常规做法是写一个转换器把 NPZ 展开成 UE Schema。算一下就否掉了：每段动画展开成 $[T, 942]$ 的稠密数组，全库规模约需 918 GB，而可用磁盘只有 317 GB。所以这条路不只是慢，是走不通。

我们改成让训练端直接读四键 NPZ、在读取时现算特征（`soma_realtime_dataset.py`）。磁盘零增长，一套格式贯穿评测与训练两段，也省掉了一层需要同步维护的中间产物。

6.2 关键取舍：特征算在哪里

「现算」听起来会把成本转嫁给训练。实测的三版做法给出了反直觉的结论。单条动画各环节成本：读 NPZ 100.5 ms、算接触 32.2 ms、算特征 125.6 ms、切片 15.1 ms。问题出在那 125.6 ms 上：逐条调用 `DualRootGlobalJoints` 时，绝大部分是核函数启动开销，不是算力。

表 6：特征计算位置的三种放法与实测结果。同一份数据、同一台机器。

做法	单条/单批成本	结果
逐条计算（batch=1 逐次调用）	125.6 ms/条	启动开销主导，不可用
整批一次算（batch=16）	6.7 ms/批 ≈ 0.4 ms/条	相对逐条约 16×
特征留在主进程，worker 只读盘	主线程阻塞 19.7 ms	预取停摆，workers=8 反而比单进程慢 3–5×
特征在 worker 内整批算	8 进程真正并行	采用方案

第三行是最容易踩的坑：加 `num_workers` 本是常识，但若特征计算留在主进程，主线程一阻塞，worker 的预取就失去意义，实测比不加还慢。这解释了为什么「先加 worker 再看」会得出错误结论。

另外两项微优化同样来自实测：

- 四元数换矩阵：自写 Shepperd 四分支法比 `scipy.spatial.transform.Rotation` 快（20 万组 54.3 vs 68.8 ms，1.27×），两者一致性到 10^{-9}（比较时对 q 与 $-q$ 取绝对内积）。
- float32 而非 float64：每条转换成本由 280.9 ms 降到 215.1 ms。模型本身以 bf16 训练，float32 输入不构成精度瓶颈。

6.3 存储表示：把旋转从矩阵换成四元数

单条 clip 的读取耗时拆解为：`np.load` 打开 1.3 ms、读 positions 10.5 ms、读 rotations 24.8 ms、root_track 0.7 ms。rotations 占 66%，因为它以 float64 的 3×3 矩阵存储：每关节每帧 $9 \times 8 = 72$ 字节，而模型用 float32、四元数只需 $4 \times 4 = 16$ 字节，两者叠加是 4.5 倍冗余。

压缩前的无损性先验证再动手：

$$\left\| q(\mathbf{R}) - q^{\star} \right\|_{\infty} \le 5.6 \times 10^{-16} \ (\text{float64 往返}), \qquad \varepsilon_{\text{pos}} \approx 1.03 \times 10^{-7} \times 0.3\ \text{m} \approx 3 \times 10^{-8}\ \text{m}, \tag{6.1}$$

即 float32 四元数相对原 float64 的位置误差约 3×10^{-8} m，比 float32 自身的表示精度还小，对训练无影响。压缩后 `dataset.json` 增加 `rotation_format` 字段标明表示方式，原视图不动、两套目录并存以便对比。

> 一个被实测否掉的方案 —— 我们也试过对已经压缩过的 NPZ 再做一次 `savez_compressed`。结果只省 9% 体积（30.4 → 27.7 MB），却慢 36 倍（168.9 vs 4.6 ms/条）。数据在 float32 + 四元数之后已经足够小，再压缩只是把 CPU 换成一点点磁盘。这一条写在这里，是为了避免以后有人「顺手再加个压缩」。

6.4 并行度：不是越大越好

契约生成与评测原本都是单进程串行，在 32 核机器上只用到 1 个核。按窗口分片给多进程后：

表 7：并行化前后对比。契约与评测都已改写为多进程版，契约字段与串行版完全一致，产物可直接给原评测器使用。

环节	串行	并行（实测）	说明
契约生成（72,933 条）	约 30 分钟	约 12 分钟（32 进程）	主进程只做汇总与写盘
全量基线评测	—	约 16 分钟（16 并发）	4 场景 × 2 基线 × 121,058 窗口

契约的串行版是「逐条读 NPZ 算接触」，30 分钟为设计文档中的估算；评测的 16 分钟为实跑记录。两者口径不同，不做加速比相除。

但并发不是越高越好。实测 32 进程同时读盘时，吞吐反而从约 6,350 条/分钟降到约 1,400 条/分钟——I/O 队列被打满，进程在等盘而非在算。16 进程配合每窗口 20 ms 休眠可以全程稳定，且不会把磁盘压到无响应。这是本章唯一一条「加资源导致变慢」的实测记录，也是我们最终采用 16 而非 32 的原因。

6.5 训练侧的运行时参数

在数据适配层就位后，训练脚本新增了四个参数。它们只影响速度，不影响数值：

表 8：新增训练参数。默认值取保守配置，需按机器上调。

参数	作用	取值依据
`--soma_views`	直接读四键 NPZ 训练视图，跳过预转换	§6.1 的 918 GB 约束
`--num_workers`	DataLoader 并行读盘进程数	本机小文件延迟受限，4–8 之后边际收益快速衰减，不建议超过 16
`--soma_window_frames`	训练窗口帧数（A 48 / B·C 72）	由 Token 预算反推，定长便于成批堆叠
`--soma_fps_group`	只训指定帧率组	120 与 30 fps 不可混批（§4.5）

`persistent_workers` 必须开启。这条是反直觉的：在 Windows 的 spawn 语义下，直觉会认为常驻 worker 与主进程争抢、应该关掉。实测相反——关闭时每个 epoch 都要销毁重建进程池，每次付 1–2 秒的 torch JIT 初始化（日志里每个 worker 一条 `triton not found`）。200 条数据跑 5 个 epoch：关闭 57 秒，开启 24.9 秒（2.3×），且开启后首轮之后每轮只要 3.7 秒。关闭时的累积开销会让长训练看起来像卡死——我们确实一度观察到 119 步后长时间无进展。

6.6 三项被实测否掉的「优化」

「优化」不等于「有效」。下面三项按常规经验都该做，实测后全部放弃，理由都在表里。把它们写出来，是为了避免后来者重走一遍。

表 9：三项被实测否掉的优化。前两项各跑了同数据的完整对照，第三项算清了内存与命中率后放弃。

方案	预期	实测	处置
bf16-mixed	Tensor Core 加速 1.5–2×	73.2 s vs FP32 71.3 s（无加速），末段 loss 1.9093 vs 1.5901（差 20%）	不采用
clip 级 LRU 缓存	省下重复读盘	80,000 次取样只覆盖 48,022 条唯一 clip，平均每条仅 1.67 次；缓存 30 条需 0.38 GB/worker	不采用
二次压缩（`savez_compressed`）	进一步缩小体积	只省 9% 体积（30.4→27.7 MB），却慢 36 倍	不采用

bf16 的两个数字都指向「不划算」：既没提速，又损失精度。原因是本链路的 GPU 并非瓶颈（此时读盘与特征计算已占主导），而 VQ-VAE 的 EMA 码本更新对半精度敏感。这项对照同时说明：「GPU 利用率低」不等于「切成半精度就能快」——瓶颈可能根本不在 GPU。

仍未做的是 `torch.compile` 与 `channels_last`。前者在 6.2 之后已不是主要瓶颈（GPU 不是限制项），后者在 Conv1d 上收益很小，两者都留待主干训练阶段再评估。因此本章不报告它们的收益。

一条附带的数据卫生检查也在这里落地：被拒绝的 168 条里大量是 `PoseAssets`、`AimOffset`、`CurveContainer` 之类的引擎姿势资产，它们靠「帧数不足」被动过滤；若某个姿势资产恰好够长，就会漏进训练集。因此新增 `audit_training_contamination.py` 按路径模式主动核查，输出可疑清单，而不是等它在训练里表现异常再回头查。

7 实验设置

7.1 冻结的评测口径

为了在多次训练之间做可比，我们固定了三件事：

- **验证集**：48 条片段（Walk / Run / Crouch 各 16 条），共 2,608 个监督 Token，随机种子 1701，manifest 与 sidecar 哈希两次记录完全相同。
- **采样协议**：每条片段使用前 4 帧来源姿态作为历史，生成其后 24 帧；不输入任何未来真实姿态。
- **对照基线**：`ReferenceGuidedPose` 在同一条来源 Actor Root 下运行，共享同一批测试片段。两者历史长度不同（24 vs 4 帧），因此这是产品路线对照，不是「只改架构」的严格消融。

7.2 指标定义

所有指标写成公式后，"为什么 Run 的 Token CE 降了 15% 而世界误差升了 4.6 cm"这类问题才有明确的讨论对象：式 (7.1) 与式 (7.2) 衡量的根本不是同一个空间。设测试片段集合为 $\mathcal{S}$，每段评估未来 $T = 24$ 帧。

关节位置误差。按片段等权平均的世界关节 RMSE：

$$\mathrm{RMSE}_{\mathrm{pos}} = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \sqrt{\frac{1}{TJ} \sum_{t=1}^{T} \sum_{j=1}^{J} \left\| \hat{\mathbf{p}}_{s,t}^{\,j} - \mathbf{p}_{s,t}^{\,j} \right\|_2^2} \quad [\mathrm{cm}], \tag{7.1}$$

其中 $\hat{\mathbf{p}}$ 由身体位姿与外部轨迹复合得到：$\hat{\mathbf{p}}_t^{\,j} = \mathbf{q}_t \odot \hat{\mathbf{b}}_t^{\,j}$。因此这个指标同时依赖姿态质量与轨迹条件，换轨迹后不可直接比较（§8.5）。

身体朝向误差。只取水平分量，避免把翻滚分量算进来：

$$\mathrm{RMSE}_{\mathrm{yaw}} = \frac{1}{|\mathcal{S}|} \sum_{s} \sqrt{\frac{1}{T} \sum_{t=1}^{T} \mathrm{wrap}\!\left(\hat{\psi}_t - \psi_t\right)^2} \quad [^\circ], \tag{7.2}$$

$\mathrm{wrap}(\cdot)$ 把角差折回 $(-180^\circ, 180^\circ]$。注意世界朝向大部分由外部 $\mathbf{q}_t$ 决定，因此这个指标上升并不等于模型没学会转向。

接触期脚速。只在来源接触标签连续两帧成立时统计：

$$v^{\mathrm{foot}} = \frac{1}{|\mathcal{P}|} \sum_{(t,t+1) \in \mathcal{P}} \left\| \mathbf{p}_{t+1}^{\mathrm{foot}} - \mathbf{p}_{t}^{\mathrm{foot}} \right\|_2 \cdot f_{\mathrm{fps}} \quad [\mathrm{m/s}], \tag{7.3}$$

$\mathcal{P}$ 是连续接触帧对集合，$f_{\mathrm{fps}} = 30$ Hz。这是式 (3.6) 左侧的离散实现，标签本身是启发式的（§4.4），因此它只用于相对比较，且样本量小（5–24 帧对）。

接缝速度误差。衡量历史末帧到生成首帧的跳变：

$$\varepsilon_{\mathrm{seam}} = \left\| \left(\hat{\mathbf{p}}_1 - \mathbf{p}_0\right) - \left(\mathbf{p}_0 - \mathbf{p}_{-1}\right) \right\|_2 \quad [\mathrm{cm/帧}], \tag{7.4}$$

右式第二项是历史段的末速。这一项下降说明衔接更平滑，但它不说明整体步态正确。

Token 交叉熵。即式 (3.4) 在验证集上的取值：

$$\mathrm{CE}_{\mathrm{token}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \Big(-\frac{1}{|\mathcal{M}_v|} \sum_{t \in \mathcal{M}_v} \log p_\theta\!\left(z_{v,t} \mid \cdot\right) \Big) \quad [\mathrm{nats}], \tag{7.5}$$

$\mathcal{V}$ 为 48 条冻结验证片段，$|\mathcal{M}_v|$ 为该片段的监督 Token 数，合计 2,608。这个量只反映"码分类得对不对"，不含解码误差——式 (3.1) 的 D_ψ 完全不在这个指标里。式 (7.5) 与式 (7.1) 之间没有单调关系，§8.3 给出了具体反例。

加速比。处理吞吐用同一批样本的端到端耗时之比：

$$S = \frac{T_{\text{base}}}{T_{\text{opt}}}, \qquad T = \underbrace{\sum_i t_i^{\text{io}}}_{\text{文件打开/stat/目录}} + \underbrace{\sum_i t_i^{\text{cpu}}}_{\text{FK 与特征}} + \underbrace{\sum_i t_i^{\text{zip}}}_{\text{压缩}}. \tag{7.6}$$

profiling 显示 t^{io} 占 19.87 s 中的 15.86 s，t^{zip} 仅 0.32 s。这就是为什么优化集中在 I/O 而不在压缩（§4.7）。

表 10：指标定义与已知口径限制。

指标	定义	口径限制
Pose Token CE	掩码位置上的 pose token 交叉熵，单位 nats	只反映 Token 分类，不含解码误差
关节 RMSE	未来 24 帧、79 骨世界关节位置均方根误差	依赖外部轨迹；轨迹不同时不可直接比
接触脚速	来源接触标签连续两帧成立时的脚部世界速度	启发式标签；样本量小（5–24 帧对）
接缝速度误差	历史末帧与生成首帧之间的速度跳变	反映衔接平滑度，不反映整体步态
朝向 RMSE	身体朝向相对来源动作的均方根误差	世界朝向大部分来自外部 Actor 旋转
时延	Python eager 前向，同步计时，含预热	不含数据准备、NNE、蒙皮、渲染、线程调度

8 结果

8.1 训练动态

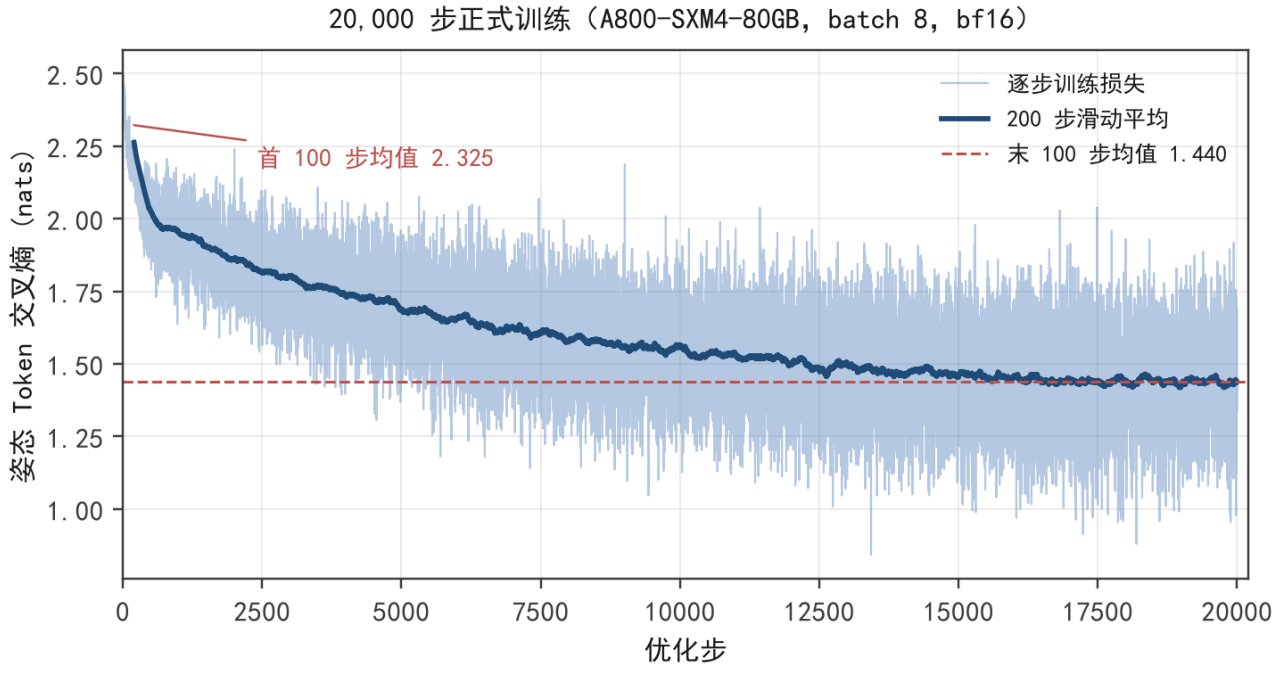


图 2：20,000 步正式训练的逐步训练损失（浅色）与 200 步滑动平均（深色）。前 100 步均值 2.325 nats，末 100 步均值 1.440 nats，虚线为末 100 步均值参考线。曲线在 12,500 步后进入平台期，末段仍有可见的步间抖动。原始数据见 `metrics.csv`（20,119 行）。

训练以 `Trainer.fit stopped: max_steps=20000 reached` 正常结束，检查点在本地与远端 SHA-256 一致（`5c41d53b…3199`）。2,000 步链式短跑记录到相似的早期下降速度，这被用来确认链路正确，而不是用来证明「更长的训练更好」——两者的调度器配置不同。

8.2 固定验证口径下的 Token 交叉熵

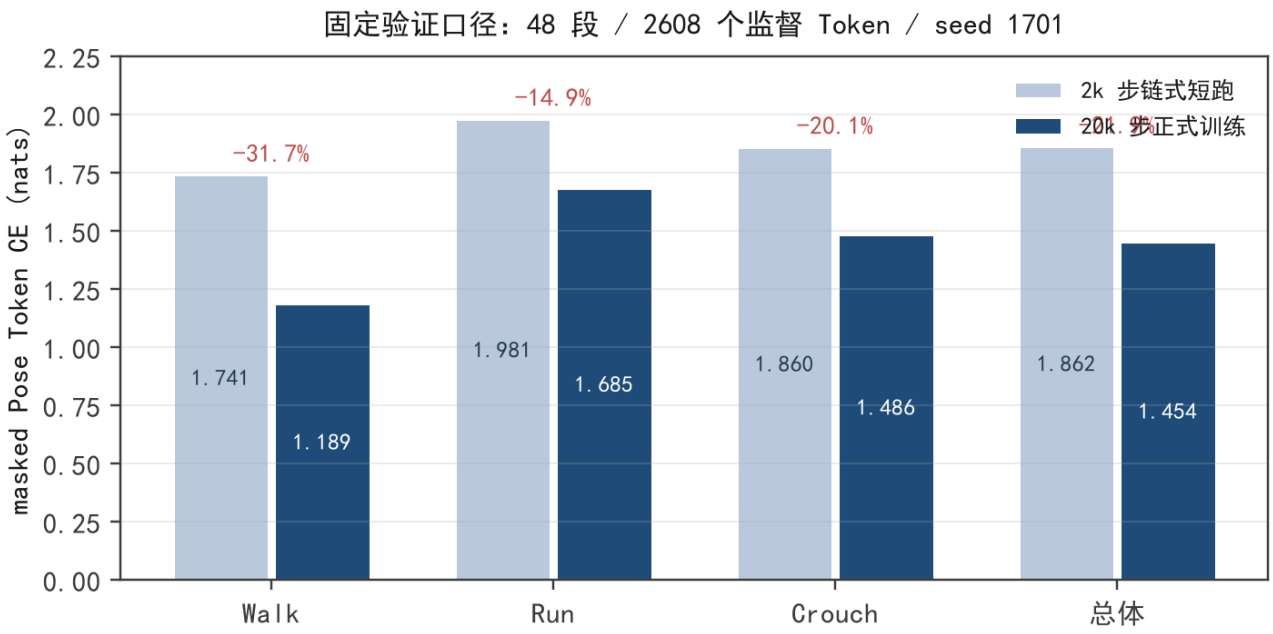


图 3：同一 48 段冻结验证集上的掩码 Token 交叉熵。柱上方数字为相对 2k 的百分比变化。 这是源 Actor Root 教师轨迹条件下的指标，只证明此固定子集的 Token 预测改善， 不代表世界姿态、接触或 UE 实时效果。

表 11：固定验证集的 Token 交叉熵（nats，越低越好）。 两次记录使用完全相同的 manifest、sidecar 哈希与索引集合。

类别	目标 Token	2k CE	20k CE	相对变化
Walk	922	1.7407	1.1887	−31.7%
Run	953	1.9809	1.6847	−15.0%
Crouch	733	1.8599	1.4863	−20.1%
总体	2,608	1.8620	1.4536	−21.9%

注：Run 的改善幅度最小（−15.0%），与 §8.3 中 Run 的世界姿态退化方向一致， 是「Token 级改善未必传导到世界姿态」的一个具体例子。

8.3 同轨迹下的世界姿态与接触

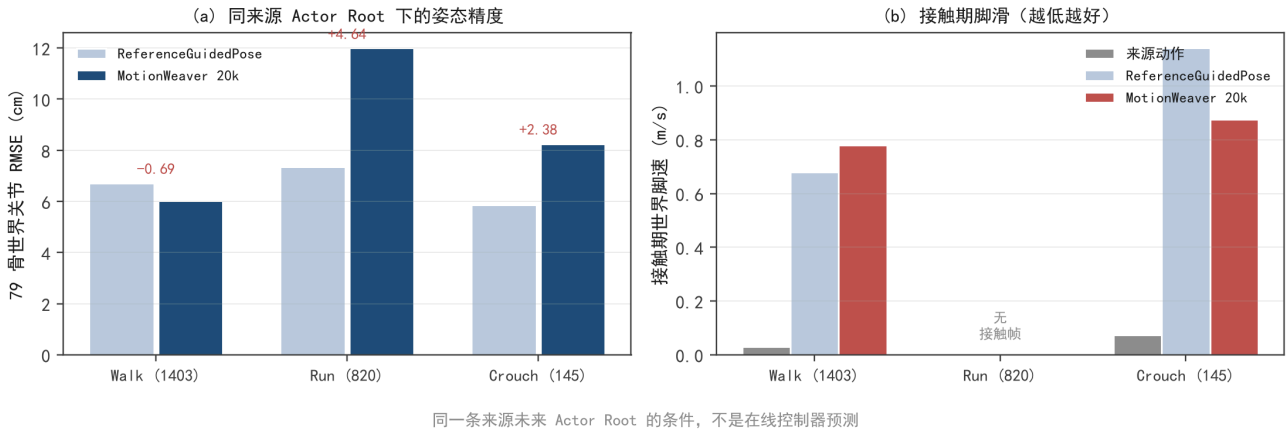


图 4：(a) 未来 24 帧 79 骨世界关节 RMSE，柱上数字为 20k 减基线之差；(b) 接触期世界脚速，灰色为来源动作的理想值。(b) 中 Run 无连续接触帧对，因此不计算该项。两组都使用同一条来源导出的未来 Actor Root，这是离线配对教师轨迹，不是玩家可提前取得的轨迹。

表 12：同轨迹循环片段的世界姿态与接触指标。RMSE 与脚速之外的列用于诊断衔接与朝向。

片段	关节 RMSE (cm) 基线 → 20k	来源脚速 (m/s)	接触脚速 (m/s) 基线 → 20k	接缝速度误差 (cm/帧) 基线 → 20k	朝向 RMSE (°) 基线 → 20k
Walk 1403	6.72 → 6.03	0.031	0.681 → 0.780	3.73 → 2.39	12.25 → 4.35
Run 820	7.36 → 12.00	—	—（**无接触帧对**）	6.86 → 2.44	7.39 → 7.07
Crouch 145	5.87 → 8.25	0.073	1.141 → 0.875	2.75 → 2.12	3.36 → 5.95

结论是分裂的：衔接更平滑（三段的接缝误差全部下降），Walk 的姿态与朝向都更好，但 Run 与 Crouch 的世界姿态显著退化，Walk 的接触脚速反而略升。来源动作的接触脚速是 0.031 / 0.073 m/s，两个模型都在 0.78 / 0.88 m/s 量级，说明脚滑问题在这条轨道上远未解决。

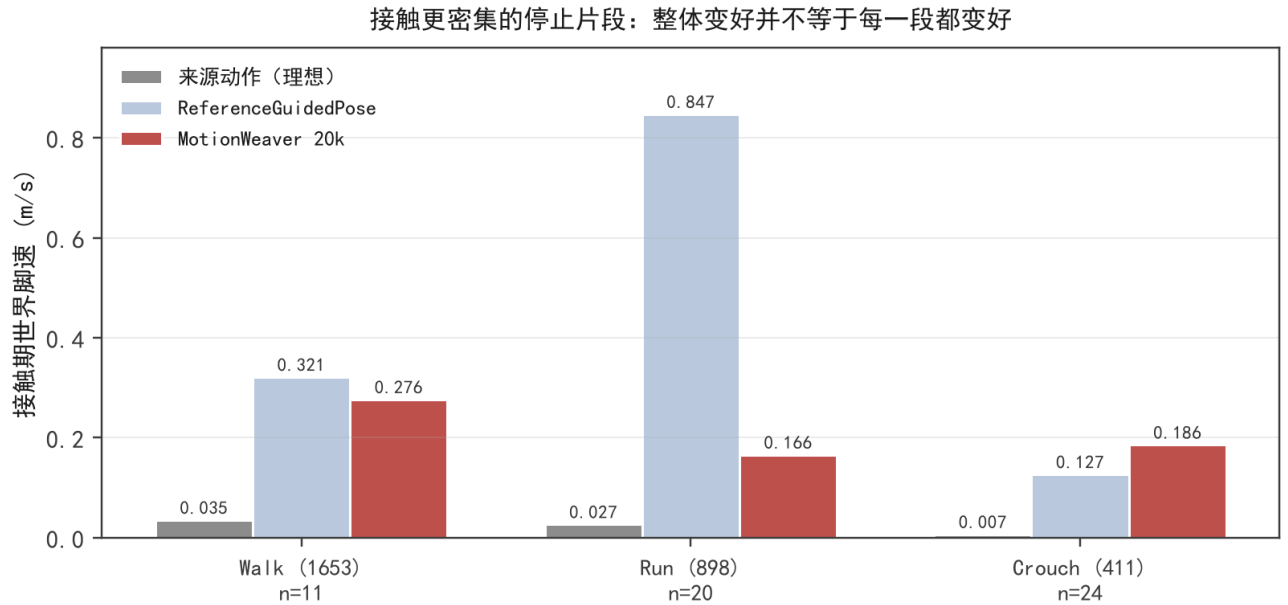


图 5：接触更密集的停止片段（接触帧对 11 / 20 / 24）。这里能看到局部改善真实存在：Run Stop 的接触脚速由 0.847 降到 0.166 m/s，Walk Stop 由 0.321 降到 0.276；但 Crouch Stop 反而由 0.127 升到 0.186。「整体变好」不等于「每一段都变好」，所以三个片段都列出来而不做平均。

表 13：接触停止片段的完整指标。停止片段的接触帧对更多，因此接触脚速比循环片段更可信；但样本量仍在 10–24 帧对的量级。

片段	接触帧对	关节 RMSE (cm)	来源 / 基线 / 20k 脚速 (m/s)	脚速 RMSE (m/s)	接缝误差 (cm/帧)
Walk 1653	11	10.28 → 9.21	0.035 / 0.321 / **0.276**	0.898 → 0.498	5.02 → 2.05
Run 898	20	10.25 → 14.13	0.027 / 0.847 / **0.166**	1.135 → 0.577	5.94 → 2.29
Crouch 411	24	5.69 → 13.42	0.007 / **0.127** / 0.186	0.400 → 0.362	4.25 → 3.21

8.4 为什么 Run/Crouch 会退化：一个可检验的解释

数据审计给出了一条线索。记第 i 条片段的 Actor 水平速度为 $v_t = \|\mathbf{q}_{t+1}[xz] - \mathbf{q}_t[xz]\|_2 \cdot f_{\text{fps}}$，统计其分位数 $Q_{0.1}, Q_{0.5}, Q_{0.9}$（米/秒，按帧汇总，排除补齐帧）：

- Walk（48,443 帧差）：p10 / p50 / p90 = **1.291 / 1.650 / 2.000**
- Run（33,734 帧差）：**1.247 / 3.750 / 5.000**
- Crouch（40,112 帧差）：**1.453 / 1.650 / 2.250**

而评估所用 CMC 轨迹的速度是 Walk 1.4、Run 4.0、Crouch 0.8 m/s。定义分布偏移量为 CMC 速度在训练分布中的分位位置，以 $Q_{0.1}$ 为下界则 Crouch 的偏移可写成

$$\kappa = \frac{Q_{0.1} - v^{\text{cmc}}}{Q_{0.1}} = \frac{1.453 - 0.800}{1.453} \approx 0.449, \tag{8.1}$$

即 CMC 的蹲伏速度比训练分布的 p_{10} 还低约 45%。相比之下 Walk 的 $\kappa = (1.291 - 1.4)/1.291 \approx -0.084$（略低于 p_{10} 之下但符号相反，说明仍在分布边缘），Run 的 4.0 落在 $[1.247, 5.000]$ 内、$\kappa < 0$。也就是说：Walk 落在训练分布低端、Run 在分布内、Crouch 0.8 明显低于训练的 p10。这解释了 Crouch 退化的一部分——模型没有见过这么慢的蹲伏位移，而且它也没有显式的 Crouch 状态输入。这是一条解释，不是一条已完成的因果证明：要证明它需要一次只改速度分布的受控实验，我们尚未做。

8.5 控制器条件探针：换一条轨迹会发生什么

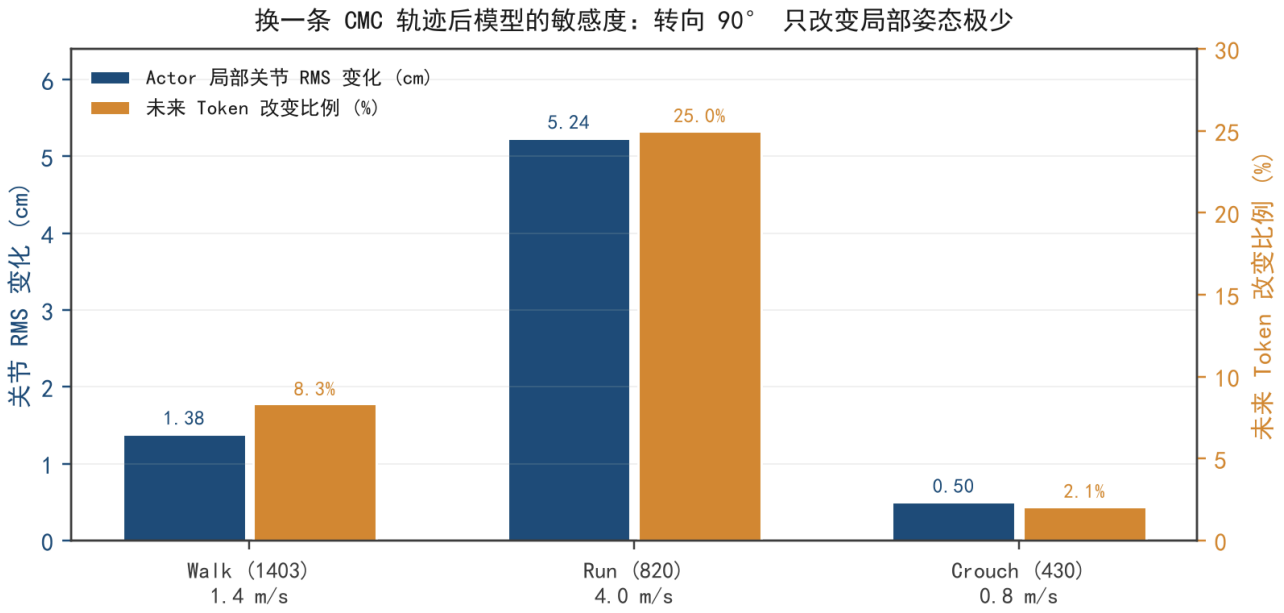


图 6：把外部轨迹从「来源导出的教师轨迹」换成「另录的 CMC 轨迹」后，模型输出的 Actor 局部关节 RMS 变化（蓝）与未来 Token 改变比例（橙）。三条片段都有约 90° 的 Actor 转向。Crouch 430 的 CMC 捕获带有蹲伏状态，但该状态没有输入模型。

敏感度用两个量刻画。第一个是 Actor 局部关节的 RMS 变化：

$$\Delta_{\text{local}} = \sqrt{\frac{1}{TJ}\sum_{t,j}\left\|\hat{\mathbf{b}}_t^{\text{cmc},j} - \hat{\mathbf{b}}_t^{\text{teacher},j}\right\|_2^2} \quad [\text{cm}], \tag{8.2}$$

第二个是离散 Token 的不一致比例（而非欧氏距离，因为 Token 是类别量）：

$$\rho = \frac{1}{L}\sum_{\ell=1}^{L}\mathbb{1}\left[z_\ell^{\text{cmc}} \neq z_\ell^{\text{teacher}}\right] \in [0,1]. \tag{8.3}$$

若模型真的对轨迹敏感，换一条转向 90° 的轨迹应当显著改变 ρ。实测 Crouch 只有 $\rho = 2.1\%$、$\Delta_{\text{local}} = 0.50$ cm，这组小数字正是"模型没响应蹲伏状态"的量化证据。

表 14：控制器条件探针。局部关节 RMS 变化衡量「模型对轨迹变化的敏感度」；Actor 转向与生成身体转向的差衡量「世界朝向变化的来源」。

片段	CMC 均速 (m/s)	局部关节 RMS 变化 (cm)	未来 Token 改变比例	Actor 转向 / 生成身体转向	CMC 轨迹中的蹲伏帧
Walk 1403	1.40	1.38	8.3%	90.0° / 94.07°	0 / 24
Run 820	3.96	5.24	25.0%	90.0° / 75.21°	0 / 24
Crouch 430	0.80	0.50	2.1%	90.0° / 93.74°	**24 / 24**

注意 Crouch 行：整段 CMC 轨迹都处于蹲伏状态，但模型输出的局部关节只变化 0.50 cm、未来 Token 只改变 2.1%，视频中「来源姿态下蹲而模型仍直立」。世界朝向的大幅变化来自外部 Actor 旋转本身，不能据此宣称模型学会了转弯或蹲伏。

8.6 上限诊断：解码器缺的是什么

为了区分「主干预测错了」与「解码器条件不足」，我们做了一个故意的 oracle 实验。固定 Token（$z_\ell^{\text{cmc}} \equiv z_\ell^{\text{teacher}}$）后，误差的分解为

$$\underbrace{\text{RMSE}(\hat{\mathbf{p}}(\mathbf{c}^{\text{ext}}=\mathbf{0}))}_{\text{零外部}} - \underbrace{\text{RMSE}(\hat{\mathbf{p}}(\mathbf{c}^{\text{ext}}=\mathbf{c}^\star))}_{\text{oracle}} = \underbrace{\Delta_{\text{dec}}}_{\text{解码条件缺失}}, \tag{8.4}$$

其中 $\mathbf{c}^\star$ 是来源未来的 pose-feature Root——推理期不可得。因此式 (8.4) 右端是上界诊断的量，不是可部署收益。若 Δ_{dec} 很小，说明瓶颈在主干；若很大，说明瓶颈在解码条件。实测 Walk 的 $\Delta_{\text{dec}} = 6.03 - 4.86 = 1.17$ cm，Run 为 0.93 cm，Crouch 为 1.77 cm（§8.6 表 15）。

表 15：解码器 pose-root oracle 诊断。Token 完全相等（已逐项校验），唯一变化是解码器的外部 Root 条件。这是未来真值泄漏的离线上界，不是可部署效果。

片段	关节 RMSE (cm) 零外部 → oracle	接触脚速 (m/s) 零外部 → oracle	脚速 RMSE (m/s) 零外部 → oracle
Walk 1403	6.03 → 4.86	0.780 → 0.970	0.769 → 0.693
Run 820	12.00 → 11.06	—	1.559 → 1.466
Crouch 145	8.25 → 6.48	0.875 → 0.437	1.168 → 0.831

关节误差可由 6.03 / 12.00 / 8.25 降到 4.86 / 11.06 / 6.48 cm，说明缺少合理的解码器 pose-root 条件是部分误差的来源。但 Run 仍停在 11 cm，且 Walk 的接触脚速在 oracle 下反而变差，因此这组数不能解释全部脚滑，也不能当作「补上这个条件就好了」的证据。

8.7 分词器码本初始化的 A/B 对照

分词器是整条链路的第一环，它的质量决定后续主干能拿到什么。这里对照一个只影响初始化、其余全部锁死的变量：码本用随机初始化还是用首批数据的 k-means 中心初始化（`kmeans_init`）。两臂各训满 3 个 epoch（42,791

步、48 帧窗口、batch 4、FP32、fps120 组 57,058 条训练样本）。两臂均以 `Trainer.fit stopped: max_steps=42792 reached` 正常结束。

需要先说清口径：本节谈的是分词器，与 §8.1–§8.6 的姿态主干是两回事。§8.1 起的所有质量指标仍来自此前在 A800 上训练的 20k 检查点；分词器这次是在本机 full-v1 全库上第一次训满 3 个 epoch，它属于数据链路已验证、主干尚未接收的那一环（§10 第 3 条）。

表 16：码本初始化 A/B。末段指最后 10% 的 4,279 步，同时给均值与中位数——原始逐步数据含 40+ 的尖峰，只看均值会被离群点抬高。

指标（末段）	A 随机初始化	B k-means 初始化	差异
总 loss（中位数）	0.4026	0.5165	A 低 28.3%
重建损失（均值）	0.1811	0.1815	持平（+0.2%）
码本 perplexity（均值）	5.2925	5.1851	A 高 2.0%
commit 损失（中位数）	0.3559	0.2247	B 低 36.9%

结论是保持随机初始化。总 loss 差 28% 而两臂分布大幅重叠，因此这是「统计上成立、幅度中等」的结论，不是压倒性优势。

一个只看数字会读错的地方。B 的 commit 损失低 37%，看起来更好；但 commit 损失衡量的是「编码器输出与所选码字的距离」——它低也可能意味着码本根本没被用起来。perplexity 曲线证实了后者：A 在第 10k 步就把码本用到 4.5，而 B 一直压在 2、直到 25k 步才开始上升。换言之，B 早期那个「极低的 commit 损失」不是学得好，而是量化器近似退化的副产物。这条提醒对所有以「损失更低」为唯一判据的实验都成立。

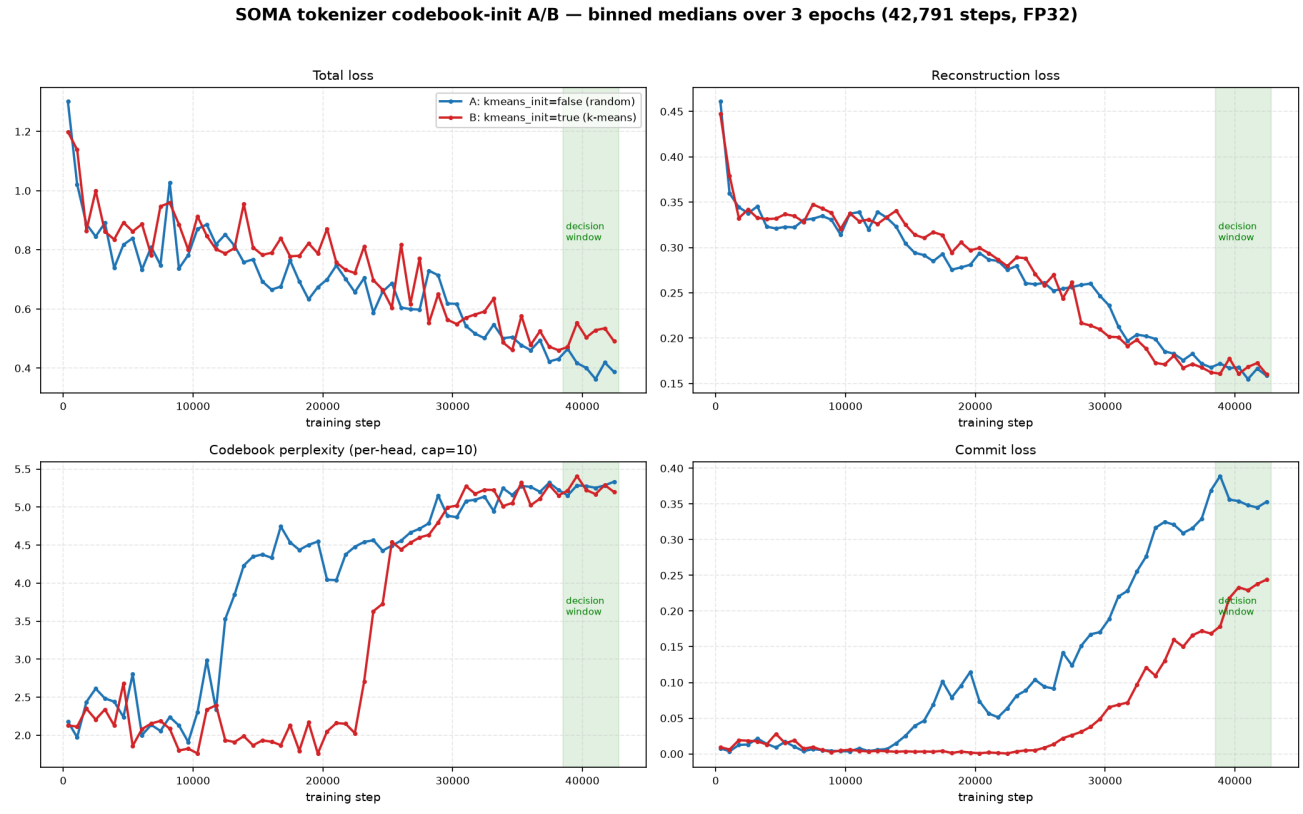


图 7：四个指标按步对齐的曲线。每点取约 700 个连续步的中位数——逐步原始数据的尖峰可达 40+，直接画只能看到噪声。绿色区间是末段 10%，即表 16 的统计窗口。perplexity 面板最能说明问题：A（蓝）在第 10k 步已达 4.5，B（红）直到 25k 步才起步。

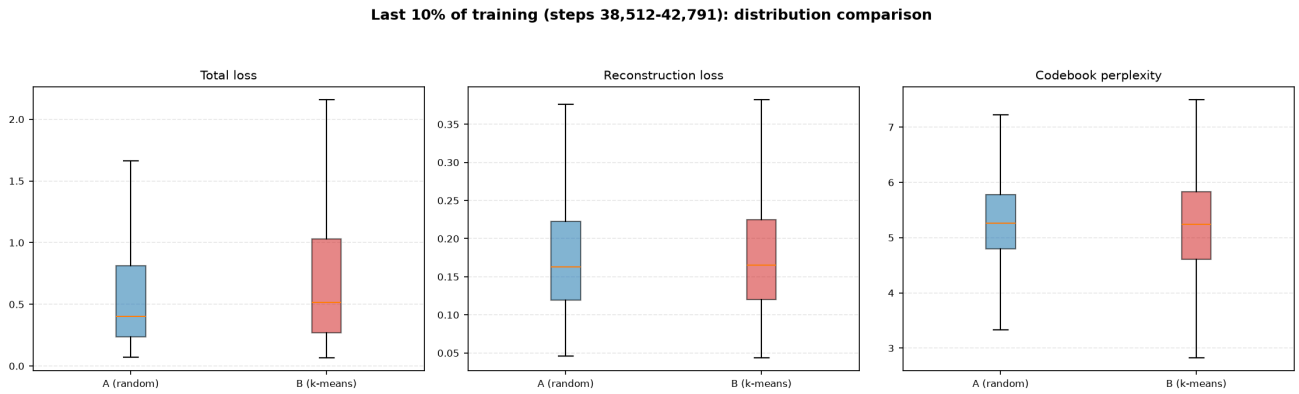


图 8：末段 10%（step ≥ 38,513，4,279 步）的分布对比。 两个箱体大幅重叠，这正是本文只主张「A 更优、幅度中等」而不称压倒性优势的依据。

> **PERPLEXITY 5.3 不是码本塌缩 —— 配置 `nb_code=1e8`、`num_heads=8`，代码按** nb-code-per-head $= \text{round}\left(2^{\log_2(\text{nb_code})/\text{heads}}\right) = 10$ **计算； 且 `calculate_per_head_perplexity: true`，日志里是单 head 的** perplexity， **上限即** 10。**因此** 5.3 **是** 53% **的单头利用率，不是「**2048 **个码字只用** 5 **个」。 真正的表达力来自** 8 **个** head **的乘积量化： `from_mh_indices_to_overall_indices` 用** 10^i **加权求和，组合数** 10^8。 **这条同时解释了为什么「调大** nb_code_per_head**」不是显然的改进—— 码字越多，每个码字被采样越少，**EMA **更新越慢，越容易出现死码**。

8.8 分词器评测：统一 benchmark 与四臂结果

评测代码原先散在三处（`training/evaluation/`、`data/tools/`、`paper/analysis/`），同一指标在不同脚本里统计口径不一致——有的取均值、 有的取中位、有的取 p90， 导致数字无法互相比较。我们把它们收进 `benchmark/`，并定下一条原则：官方已有的不重写，我们只做补集， 但对外是一份报告。

表 17：评测体系的三组分工。指标原语（`metric()` / `rms()`）直接 import 官方的 `inference.profiling.evaluate_unreal_streaming`，不重新实现。

组	内容	来源
A 组	10 **个训练指标**（perplexity / commit / recons* / joint_vel / skate_contact / loss）	**官方 `motion_vqvae.py:336-352` 训练时写入 `metrics.csv`， 本报告只读不算**
B 组	**关节** RMSE（**厘米**）、feature L1、**自校验误差**	**我们的补集：官方脚本只吃** UE Schema， SOMA **四键** NPZ **需另写**
C 组	Hold / Lerp **零成本基线**	**引用已冻结的基线报告，不重算**

表 18：四臂的 A 组指标（训练集口径，末段 10% 均值）。`l_recons_pose_global` 与两条 `l_recons_root_*` 在四臂上恒为 0.0000——根运动与全局姿态的重建完全精确，误差集中在局部姿态。

指标	BC120 (B+C, dt3)	A120 (A, dt2)	B120 (kmeans, dt2)
`l_recons_pose`	0.3032	0.1811	0.1815
`l_commit_pose`	0.0050	0.5844	0.5519
`l_joint_vel`	0.1798	0.2397	0.3329
`l_skate_contact` (m/s)	0.3911	1.1499	1.1103
`perplexity_pose`	5.6350	5.2925	5.1851
`loss`	0.6668	0.6838	0.8694

一个必须写明的陷阱：`l_recons_pose` 不可跨 `down_t` 比较。把 A 组与 B 组并排看会得到一个矛盾：A120(dt2) **的 `l_recons_pose` 最低（**0.1811**），关节误差却高达** 208.98 cm；BC120 **的** loss **较高（**0.3032**），关节误差反而最小（**105.33 cm**）。原因是该损失在归一化特征空间里计算，而归一化用的 `mean/std` 随 `frames_per_token` 变化。因此** loss **只能在同一个 `down_t` 内比较：**A120(dt2) 0.1811 **与** B120(dt2) 0.1815 **几乎相等，这与** §8.7 **得出的「**kmeans_init **无优势」一致。**

表 19：B 组（我们的补集）。自校验是官方 `motionweaver_preview.py:416` 的门槛——逆变换必须能把原始输入还原到 0.03 m 以内，否则重建指标无意义。

模型	feature L1	关节 RMSE (cm)	自校验 (m)
BC120 (B+C, dt3)	0.7233	105.33	0.000000
A120 (A, dt2)	0.7152	208.98	0.000000
A120 (A, dt3)	0.6572	121.04	0.000000
B120 (A, kmeans, dt2)	0.7824	308.09	0.000000

自校验全部为 0.000000 m，说明「特征 → 关节位置」的转换链路完全自洽，表中的差异来自模型本身而非评测代码。但关节 RMSE 绝对值很大（105–308 cm），说明即使特征级损失已经较低，还原到关节位置后误差仍不可接受。

脚滑是唯一可直接与基线对比的指标，因为 `l_skate_contact` 的单位是 m/s。BC120 **的** 0.3911 m/s **已优于** Lerp **基线的** 0.545 m/s（21× **源动作**）。**但两者口径不同：基线是补间生成的结果，这里是** tokenizer **重建的结果，不能据此声称「模型补间能力超过** Lerp**」。**

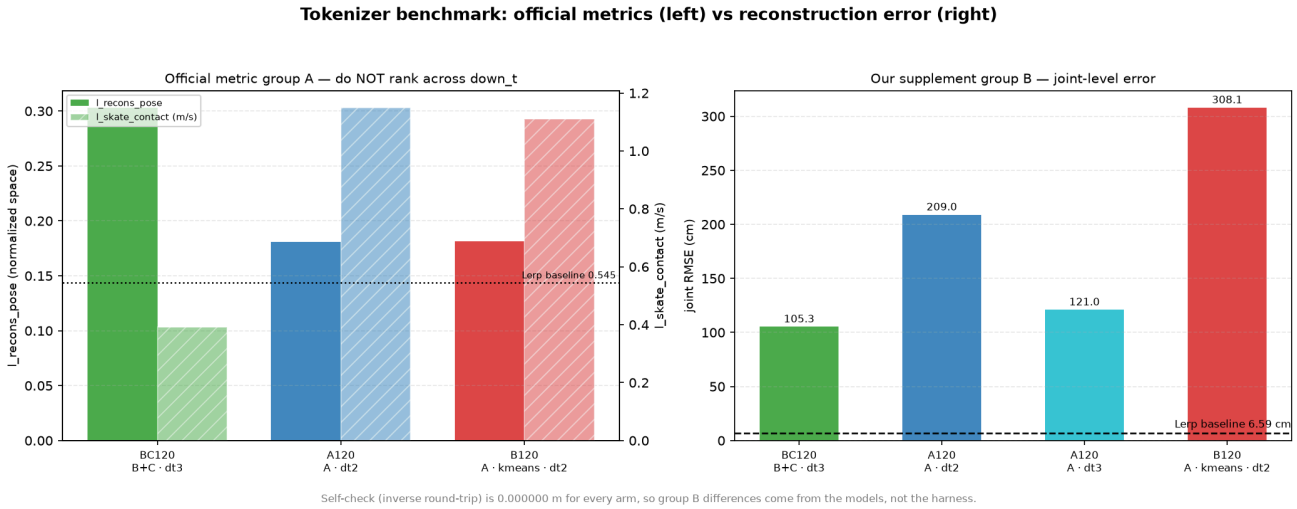


图 9：四臂在 A 组（训练指标，左）与 B 组（重建误差，右）上的对比。 A 组的 `l_recons_pose` 不可跨 `down_t` 排序（见正文）， 因此左图只用于同 `down_t` 内的比较。

表 20：Python eager 前向的同步计时（RTX 4070 Laptop，每条 1 次未计时预热， 计 3 条 × 3 次同步前向）。不含数据准备、UE NNE、蒙皮、渲染与线程调度。

条件	P50 (ms)	P95 (ms)	4 帧预算 @30 Hz (ms)	是否在预算内
配对教师轨迹	18.24	21.04	133.33	**是**
CMC **轨迹**	17.78	19.95	133.33	**是**

作为工程参照：同仓库的 UE 实验插件在旧 VQ-VAE 重建任务上报告过 6.07 ms 平均 / 10.45 ms P95 的同步推理耗时，并伴随 9.06 cm 平均关节误差。 两个数字属于不同任务与不同模型，不能相加或互相替代。

9 损失函数与评测口径：一次被口径掩盖的失败

本节记录一条完整但结论为否定的实验线：我们试图让训练目标直接对齐验收指标， 把米制关节误差写进损失，结果在 17 组同步数对照下确认该损失没有可测量的作用。 但更有价值的东西不是这个否定结论，而是中途发现的一件事： 在此之前， 训练与评测喂给模型的不是同一个分布，而我们此前所有长训练的结论都建立在这个口径差之上。

9.1 动机：训练在优化的量，不是验收在量的量

分词器的训练损失是 941 维归一化特征上的 SmoothL1，量级约 0.26；而验收指标是把重建结果还原成世界空间骨架后的 关节位置 RMSE（厘米）。两者之间隔着归一化的 `mean/std`、`local⇔global` 的两级表示转换， 以及根运动积分。§8.8 已经记录过这个断裂的一个侧面： `l_recons_pose` 最低的臂（A120, dt2, 0.1811） 关节误差反而高达 208.98 cm。

于是有了一个自然的想法：既然验收看米制关节误差，就把它写进损失。 我们在解码器输出上先还原到世界空间关节位置，再对预测与真值算 SmoothL1，以系数 `joint_pos_loss_coeff` 加入总损失。实现时先修掉一个致命错位： 原 `loss()` 里 预测走 `local_motion_rep.inverse`、真值走 `global_motion_rep.inverse`， 在当前 `pose_vqvae_motion_rep: local` 配置下两者实测相差 120.50 cm， 直接相减等于在罚两个坐标系之差。修正后往返精确到 1.2×10^{-7}（特征）与 1.0×10^{-6} cm（关节）。

另有两个前置疑问必须先用实验堵死，否则后面任何否定结论都可能只是「没接对」：

表 21：在花训练时间之前先排除的两个实现性怀疑。两项都是否定性结论—— 即这两个解释都被证据排除，因此后续结果只能从训练动力学本身去找原因。

怀疑	验证方式	结果	判断
训练口径与评测口径不同构	**同一检查点分别按训练口径（`is_normalized=True` 链路）与评测口径还原关节，比较误差**	72.14 对 72.14；77.32 对 77.32，**精确相等**	**排除。gt 之间 64.07 cm 的整体旋转属于等距变换，误差不变**
梯度没有足量传到网络	**只对关节位置损失反传，测 encoder / decoder 梯度范数**	encoder 1.17、decoder 20.39，**比重建损失大 3.6 倍**	**排除。梯度既存在量级也合理**

9.2 反常：训练损失在降，评测却在同一步数上暴涨

全库（71,127 条 fps120）上跑两组完全相同的配置，只差 `joint_pos_loss_coeff`（0 与 0.1），各 40,000 步，每 10,000 步存档一次。按常理两条曲线应当接近。实际拿到的却是：

表 22：全库训练在 30,000 步处的断崖式退化。同期训练损失并未发散，`l_recons_pose` 仍在缓降（0.330 → 0.285），因此不能用「过拟合」或「训练崩了」解释。评测口径：`evaluate_all(ckpt, rows=(8,), clips=16)`，16 条 B+C 片段，自校验均 $\approx 3.78 \times 10^{-7}$ m（门槛 0.03）。

步数	无关节损失	关节损失 0.1	feature L1	说明
10,000	105.53	105.62	0.787	**两组都还在「能看」的范围**
20,000	108.39	106.28	0.786	**平稳**
30,000	310.23	298.45	0.972	**两组同时断崖，与损失无关**
40,000	330.13	366.07	0.994	**继续恶化**

同期 `perplexity_pose` 由 4.26（26k）升到 6.08（30k）、`l_commit_pose` 由 0.0015 升到 0.0179（约 12 倍），提示码本侧发生了状态变化，但两条独立配置在同一步数一起塌陷，更像是某个与配置无关的量出了问题。

先排除「只是某个场景难」：把 30,000 步的检查点按场景分开评测，四个场景同时变差（A 277.80 / B 210.52 / C 180.42 / D 186.05 cm）。**这排除了数据混合比例、场景分布之类的解释。接下来要回答的是：如果模型没坏，那是不是我们给它看的东西不一样了？**

9.3 根因：训练与评测喂了不同的归一化分布

沿着「训练路径 vs 评测路径」逐段对照，问题落在 `training_step` 里的一行参数上。两条路径在同一批数据（16 条 × 72 帧）上送进 `pose_net` 的局部身体特征（937 维 body 通道）统计量如下：

表 23：同一批数据、三种转换链得到的 `local body` 统计量。第三行是把原始特征先 `normalize` 再按「输入已归一化」进入转换链——也就是理论上正确的写法。用它作参照，可以判定哪一条路径是对的。

转换链	body std	与「理论上正确」的 max $\lvert\Delta\rvert$	判断
训练路径（旧）：`change_first_heading(..., is_normalized=True)` 直接吃 collate 的原始特征	0.9669	862.40	**错。把未归一化值当成 z-score 用了**
评测路径：`global_to_local(..., is_normalized=False)`	0.5845	6.85×10^{-6}	**对。等价于理论上正确的写法**
理论上正确：先 `normalize`，再 `is_normalized=True`	0.5845	0	**参照**

「判断」列的判定依据是第三行：与它相差 10^{-6} 量级的路径是对的，相差 862 的那条是错的。误差不是随机噪声，而是系统性错位。

根因是两条数据入口对 `batch["motion"]` 的归一化状态恰好相反，而 `training_step` 把 `is_normalized` 写死成了 `True`：

表 24：两条数据入口产出 `batch["motion"]` 时的归一化状态。二者相反，因此这个开关不能靠默认值猜，必须按数据来源显式声明。

入口	产出方式	归一化状态	应取的 `is_normalized`
`--dataset`（`UnrealMotionDataset`）	`build_unreal_training_set.py:204` 落盘前已做 $(x-\mu)/\sigma$	已归一化	`true`
`--soma_views`（`SomaRealtimeDataset`）	`soma_realtime_dataset.py:389` 用 `to_normalize=False` 产出原始特征	未归一化	`false`

这解释了一个此前被误读的现象：20,000 步之前的评测数字其实是假的「好」。那时模型基本只输出一个与输入近乎无关的常量姿态——该常量姿态本身就能拿到 51.73 cm，而 20k 的重建结果与输入特征的相关系数只有 −0.083。因为两条路径「一致地差」，差距被掩盖了。一旦模型真正开始拟合训练侧那份（被错位的）分布，评测侧的输入就成了分布外样本，关节误差从 108 cm 跳到 310 cm。所以这不是过拟合，也不是码本坏了，是训练目标与评测目标自始至终不是同一个量。

表 25：被这个口径差污染、因而全部作废的既有结论。它们都建立在 ≥ 30,000 步的检查点上。

旧结论	当时的证据	作废原因
「全库训练到 40k 会崩」	30k = 310 cm、40k = 330 cm	是口径错位，不是数据量或过拟合
「熵正则在 40k 上带来 5% 改善」	有熵 103.29 vs 无熵 108.76	30k 后数据不可信，差异落在被污染的区间
「关节位置损失使结果变差 4.8%」	108.23 vs 103.29	同上；且两组都在崩溃点之后

9.4 修复：一个显式的开关，而不是一个默认值

修法不是把 `True` 改成 `False`——那只是把同一类错误换到 `--dataset` 入口上。正确做法是把「输入是否已归一化」提升为由数据来源决定的事实：新增 `inputs_are_normalized` 参数，由训练入口按实际数据来源写入

```
conf.model.args.inputs_are_normalized = not bool(getattr(args, "soma_views", None))
```

而不是让 `motion_vqvae.py` 去猜。这个改动的价值可以用一条曲线衡量：

表 26：修复前后的同口径对比。修复前最好的成绩来自 B+C 子集的 10k 检查点；修复后在同一评测口径下，全库 15k 即达到 83.85 cm。这是本项目至今最大的一次单点收益，来源不是任何损失项，而是把口径对齐。

配置	joint RMSE (cm)	相对修复前最好	备注
修复前最好（B+C 10k）	94.54	—	此前所有报告的引用基线
修复前（全库 40k）	330.13	+249%	断崖后退化
修复后（全库 15k）	83.85	−11.3%	且 30k 之后不再崩
修复后（全库 100k）	85.59	−9.5%	稳定性成立

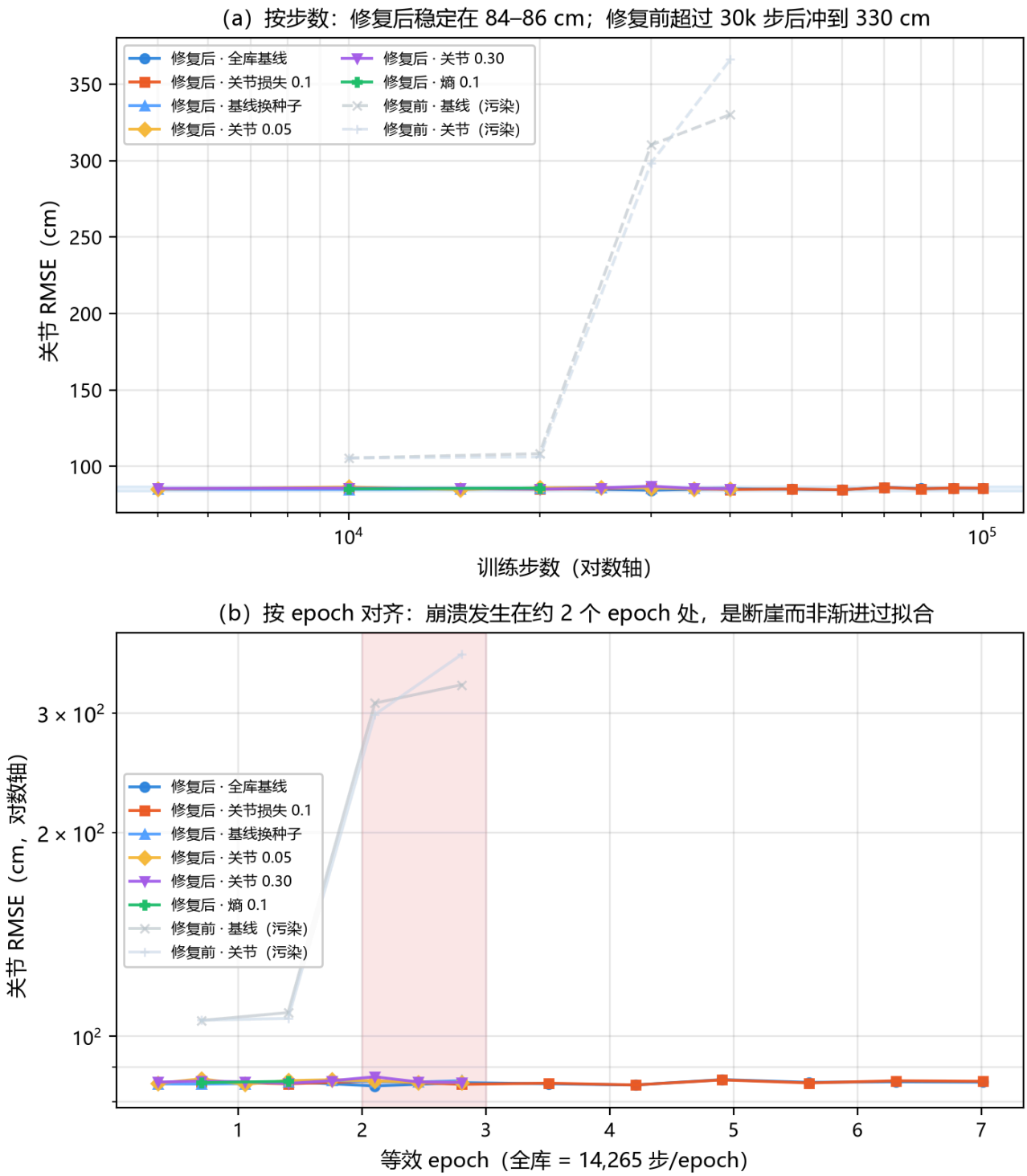


图 10：全部检查点的关节 RMSE。左图按步数、右图按等效 epoch 对齐。浅灰两条是修复前的（被污染）曲线，30k 后直接飞到 300 cm 以上；修复后的六条全部压在 84–86 cm 的带子里。右图说明崩溃点落在约 2 个 epoch 处，与「数据量不足导致过拟合」的直觉不符——过拟合应当是渐进的，这里是断崖。

为了让这次修复本身可被回归测试守住，我们补了一组测试，并且做了变异检查：把 `is_normalized` 写死成任一值、把开关的读取去掉、让训练入口不再按数据来源设置，四种人为破坏全部被测试捕获（4/4）。只写测试不做变异检查是不够的——本仓库此前出现过「10 个测试全绿，但把熵项整个删掉依然全绿」的情况。

9.5 结论一：关节位置损失没有可测量的作用

口径修好之后，才有资格回答最初的问题。两组配置在全库上各跑 100,000 步，每 10,000 步存档，得到 10 个同步数配对点；另在 40,000 步处加扫 0.05 与 0.30 两个系数端点，并跑一个同配置换种子的基线来标定噪声底。

表 27：100,000 步逐点配对。Δ = 关节损失 0.1 − 基线。 十次里只有四次更优，且幅度全部小于种子噪声。

步数	基线	关节损失 0.1	Δ	Δ%
10,000	85.43	85.68	+0.25	+0.30%
20,000	85.69	85.09	−0.60	−0.70%
30,000	84.50	85.89	+1.39	+1.65%
40,000	85.43	84.94	−0.49	−0.57%
50,000	85.10	85.27	+0.17	+0.20%
60,000	84.76	84.77	+0.01	+0.01%
70,000	86.31	86.23	−0.08	−0.09%
80,000	85.53	85.35	−0.18	−0.21%
90,000	85.68	85.98	+0.30	+0.34%
100,000	85.59	85.84	+0.26	+0.30%

Δ 均值 +0.104 cm、中位 +0.092 cm；关节损失更优的点为 4/10； 符号检验双侧 $p = 0.7539$。

表 28：系数扫描（40,000 步同步数）。把系数放大六倍（0.05 → 0.30） 没有出现任何单调趋势，这是「调系数没调对」这一解释的反证。

系数	joint RMSE (cm)	Δ vs 基线均值	判断
0（基线，两种子）	85.89	—	基准
0.05	85.49	−0.39	落在噪声内
0.10	85.65	−0.23	落在噪声内
0.30	85.48	−0.41	落在噪声内

表 29：本节的判据链。三条相互独立的证据都指向同一个方向， 且噪声底本身由实验测得，而不是估计的。

检验	值	读法
效应量	+0.10 cm	略微更差，但幅度可忽略
种子噪声底（同配置换种子，逐 step）	均值 0.56 / 最大 1.19 cm	效应比噪声底小约 5 倍
符号检验（10 个配对点）	$p = 0.7539$	完全无显著性
系数单调性（3 个端点）	无趋势	不是「系数没调对」
步数饱和（10k → 100k）	85.43 → 85.59	继续加步数无收益

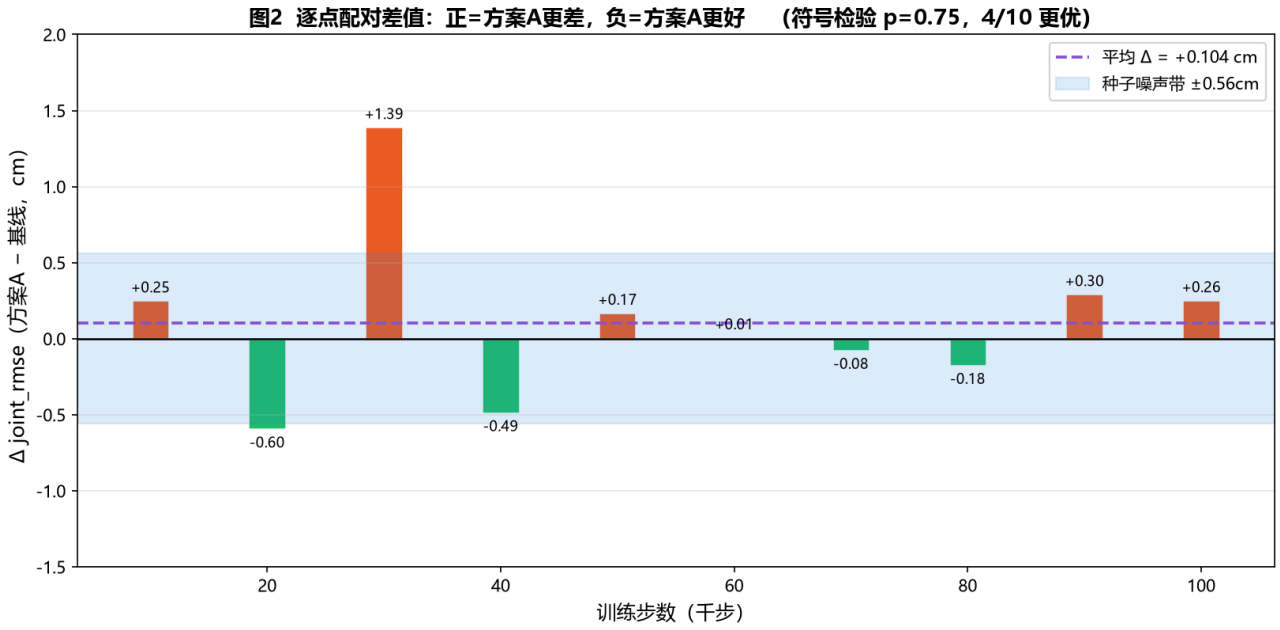


图 11：Δ = 关节损失 0.1 − 基线，正值表示关节损失更差。 柱子在零线两侧随机跳动，平均 +0.10 cm（紫虚线），除 30k 一根 +1.39 外 全部落在浅蓝色的 ±0.56 cm 种子噪声带内。如果损失有效，柱子应当整体压到零线以下。

顺带复核了另一个曾经「看起来有效」的正则项——码字使用熵。在口径修好后重测， 10,000 **步处** 85.42 **对** 85.43、20,000 **步处** 85.83 **对** 85.69，**同样落在噪声内。 此前「熵在** 40k **上有益** 5%」**的判断，正是被** §9.2 **那个断崖污染出来的假象。**

9.6 结论二：真正的瓶颈是码本塌缩

口径修好之后还暴露出一件比损失设计更根本的事： `perplexity_pose` **在** 20,000 **步后 锁死在 1.000。**

表 30：码本使用情况随时间。perplexity 降到 1.000 意味着 每一批里所有 token 都被量化到同一个码字，码本容量 `nb_code` = 32^8 几乎完全闲置，分词器实际上退化成了一个自编码器。

区间（步）	l_recons_pose	perplexity_pose	状态
0 – 10k	0.2611	1.876	**仍在收敛**
10k – 20k	0.2592	1.190	**快速塌缩**
20k – 30k	0.2613	1.000	**完全塌缩**
30k – 40k	0.2593	1.000	—
40k – 50k	0.2576	1.000	**重建几乎不再改善**



图 12： `perplexity_pose` **随训练步数的变化**（2,000 步分桶中位数）。 20,000 步之后一直贴在 1.0 的红线上。这个事实解释了 §9.5 的否定结论： **码本里没有可供优化的信息**，在 941 维特征的平均损失上下压 0.1 cm 是做不到的。

这也和 §8.7 **的旧结论对上了：那时我们看到「码本容量** 8 → 128 **帧，关节误差只变** +0.1%」， **当时的解读是「容量假设被证伪」。现在的看法更准确——容量不是没用，而是根本没被用起来。 两件事需要放在一起读：容量实验说明加大码本无效，**perplexity **说明原因是用不上。 真正的下一步是约束「太少用」而不是「不够大」，例如在量化器层面约束每批的码字占用分布， 而不是继续在损失项上做加法。**

9.7 方法论上的经验：四个差点写错的判断

§9.3 **之外，这条实验线还留下四条可直接复用的教训。它们都不是「指标算错了」， 而是在错误的前提上得出了看起来自洽的结论。**

1. **「训练损失降了」不能推出「泛化变好了」。 关节位置损失在训练窗口上的分段均值确实降了** 32%（0.4470 → 0.3052）， **而评测口径下关节** MAE **反而** +4.0%。**前者衡量的是拟合，后者衡量的是泛化， 这条线上两者方向相反过不止一次。**
2. **对比必须同步数，且步数要换算成 epoch 再看。** B+C **子集每个** epoch **是** 3,372 **步，跑到** 40,000 **步相当于** 11.9 **个** epoch， **早已越过过拟合点；而全库是** 14,265 **步一个** epoch，40,000 **步只有** 2.8 **个** epoch。 **同样叫「**40k**」，一个在过拟合区、一个还在欠拟合区。此前的「**40k **对比」正是这样失效的。**
3. **用单步值判断训练进展。** `l_joint_pos_m` **的末行单步值曾是** 0.0996，**据此得出「已达标」； 而末** 2,000 **步的中位数是** 0.2215，**标准差** 0.3105——**单步抖动比信号本身还大。 本节的结论一律基于分桶中位数或整点检查点。**
4. **没有噪声底就不能宣称「改善」。 在标定种子噪声之前，**0.05 **与** 0.30 **两个端点看起来都「改善了** 0.4 cm」， **而这个数字比同配置换种子造成的差异还小。任何** A/B **结论都必须先回答： 这条曲线上的自然抖动有多大？**

这四条的共同点是：它们都让一个错误的结论看起来很有道理。 因此本报告在 §9.2、§9.5、§9.6 **三处都刻意先写出「本来以为会怎样」， 再给出实测——差异本身就是这部分最重要的结果。**

---

10 讨论、局限与缺陷清单

10.1 这份报告能支撑什么

- Tokenizer 加 Pose Token 的两段式链路在本仓库跑通：数据、训练、检查点、离线推理、视频导出全链路可复现。
- 固定验证口径下，20k 训练相对 2k 显著改善 Token 预测（−21.9%），且该改善在三个类别上方向一致。
- 数据管线在保持逐数组等价的前提下把处理吞吐提到 1.69–1.80 倍，且有 24 次配对的哈希证据。
- 时延在 Python 前向层面有约 6 倍余量（P95 21 ms 对 133 ms 预算）。

10.2 这份报告不能支撑什么

表 31：显式的能力边界。左列是被问过的问题，右列是当前证据状态。

问题	当前状态
玩家可操控时姿态与位移一致吗	未验收。尚无 UE NNE / AnimGraph 接入、无蒙皮闭环、无真实玩家输入。
能可靠蹲伏吗	不能。CMC 探针中蹲伏状态未输入模型，模型保持直立。
脚滑解决了吗	没有。循环片段接触脚速 0.78–0.88 m/s，来源为 0.03–0.07 m/s。
Token 改善等于姿态改善吗	不等。Run 的 CE 降 15%，世界 RMSE 却由 7.36 升到 12.00 cm。
两种模型是严格消融吗	不是。历史长度 24 vs 4 帧、架构不同、参数量不同，属产品路线对照。
时延满足 UE 实时预算吗	部分。Python 前向有余量，但 UE 侧全过程未测。
SOMA 全库都训了吗	分词器已训（§9）。全库 71,127 条 fps120 上跑到 100,000 步，四场景 A/B/C/D 齐全；但姿态主干仍未在 full-v1 上开训，§8 的姿态质量指标仍来自 UE 数据上的 20k 检查点。
码本被有效使用了吗	没有。`perplexity_pose` 在 20,000 步后锁死 1.000，每批所有 token 量化到同一码字，分词器实际退化为自编码器（§9.6）。
关节位置损失能降低验收误差吗	不能。17 组同步数对照下效应 +0.10 cm，小于种子噪声 0.56 cm，符号检验 p=0.75（§9.5）。

10.3 方法论上的经验

这次工作中最值得记录的不是某个指标，而是三处「差点把结论写错」的地方：

1. 把 Actor 相对的骨盆特征当成 CMC Root。后果是把「模型跟随控制器」这一命题的验证污染了。修复方式不是改解释，而是重建 sidecar 并逐字节复核。
2. 把 `down_t` 当成可共享的超参数。它在源码里就是卷积层数，层数不同则权重不可共享。方案里「一套码本服务双帧率」的结论因此被推翻。
3. 把「接触候选」当成接触真值。启发式标签能用于相对比较，但一旦用来宣称「脚滑已解决」就越界了。

共同点是：指标本身没错，错的是指标所依附的对象身份。因此本报告把每个数字都附上它所属的条件（教师轨迹 / 控制器轨迹 / oracle），并在表 10 与表 31 中显式列出不可跨越的边界。

11 结论与下一步

MotionWeaver 已经建成一条可追溯的数据与训练链路：两套骨骼数据汇入同一目标骨架，分词器与姿态主干可分离训练与替换，评测口径被冻结并可复现，训练链路已可直接读取四键 NPZ 训练视图（§6），20,000 步训练在

Token 层面给出确定的改善。同时，世界姿态、接触质量与控制器响应三项指标 明确未达标，我们据此把当前检查点定位为可复现的基线，而不是可用产品。

下一步按依赖顺序排列：

1. **补齐在线可得条件**。把 Crouch / Gait / RotationMode 状态，以及 CMC 实际提供的 短时轨迹作为显式输入与掩码，替代当前「用来源未来 Root 当条件」的做法。
2. **修正解码器条件**。§8.6 表明 pose-root 条件是部分误差来源； 需要设计一个推理期可得的替代量（例如模型自预测的骨盆高度先验）， 并证明它不是变相的未来泄漏。
3. **把 SOMA 数据接进主干训练**。数据侧已完成（72,933 条、99.77%）， 分词器已在全库 fps120 组上训满 3 个 epoch（42,791 步， §8.7 给出码本初始化的 A/B 结论）；但姿态主干尚未在 full-v1 上开训， §9 的质量指标仍来自此前 20k 检查点。这是当前最大的缺口。
4. **完成 UE 闭环**。NNE 导出、AnimGraph 重建、动画线程推理、蒙皮播放，以及 4 帧步长加播放延迟下的可操控体验，都需要在真实场景里验收。
5. **用程序化模块兜住确定性约束**。按表 8 的分工接入 Warping 与 Foot IK， 并实测回退阈值是否合理。

每一项目标都要同时报告 Token CE、世界姿态、接触脚速与 UE 实测四项， 不接受只报其中一项的「通过」。

附录 A 复现

```
# 0. 环境（Windows，Python 3.12）
cd MotionWeaver
.\.venv\Scripts\python.exe --version

# 1. 重新生成本报告的全部插图（直接读取仓库内证据文件）
.\.venv\Scripts\python.exe paper\generate_figures.py

# 2. 由 HTML 生成 PDF（headless Chrome，A4，无页眉页脚）
powershell -ExecutionPolicy Bypass -File paper\build-pdf.ps1

# 3. 数据处理（示例：SOMA 全库批次，参数写在一行以避免续行符）
.\.venv\Scripts\python.exe -u -m data.tools.process_soma_dataset --data-root E:/MotionWeav

# 4. 训练（A800，20,000 步，从零初始化 Pose Backbone）
python -u -m training.pretrain.train_unreal --model pose --dataset ../dataset --actor_root
```

附录 B 证据文件索引

文件	内容
`docs/experiments/2026-09-28-motionweaver-a800-actor-root.md`	20k 训练与本地验收的完整记录
`…/assets/…/metrics.csv`	20,119 行逐步训练日志
`…/assets/…/val_token_ce_2k.json` 、 `…_20k.json`	固定验证集的 Token CE 与索引集合
`…/assets/…/motionweaver_20k_acceptance_summary_20260928.json`	本报告 §8.3–§8.7 的数值来源
`…/assets/…/config.yaml`	训练配置（含 VQ 哈希、调度器与条件 profile）
`docs/experiments/2026-10-18-soma-processing-speed.md`	吞吐优化与 24 次等价核对
`docs/soma-training-execution-plan.md`	双帧率对齐、场景划分、窗口与 Token 预算
`inference/profiling/pose-reference-lab.md`	硬参考等式约束投影的对照结果
`docs/soma-tokenizer-joint-error-options.md`	§9 的完整实验记录与判别链（口径修复、系数扫描、码本塌缩）
`docs/assets/plan-a-final/plan_a_full_table.csv`	§9.5 全部检查点的原始指标（rmse / p95 / max / feature_l1 / 自校验）
`docs/assets/plan-a-final/WEIGHTS_MANIFEST.md`	§9 各 run 的配置、步数、最优指标与权重位置

参考文献

[1] NVIDIA GEAR Lab. *GR00T-WholeBodyControl*. GitHub repository, 2025. github.com/NVlabs/GR00T-WholeBodyControl

[2] NVIDIA. *MotionBricks: real-time versatile motion generation for humanoid robots*. Project page, 2025. nvlabs.github.io/motionbricks

[3] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] P. Esser, R. Rombach, and B. Ommer. Taming transformers for high-resolution image synthesis. In *CVPR*, 2021.

[5] C. Guo, S. Zou, X. Zuo, S. Wang, W. Ji, X. Li, and L. Cheng. Generating diverse and natural 3D human motions from text. In *CVPR*, 2022.

[6] G. Tevet, S. Raab, B. Gordon, Y. Shafir, D. Cohen-or, and A. H. Bermano. Human motion diffusion model. In *ICLR*, 2023.

[7] J. Zhang, Y. Zhang, X. Cun, S. Huang, Y. Zhang, H. Zhao, H. Lu, and X. Shen. T2M-GPT: Generating human motion from textual descriptions with discrete representations. In *CVPR*, 2023.

[8] G. Chen, Y. Mu, Z. Wang, Y.-H. Wu, S. Lin, C.-C. Hsu, and Y.-J. Zhang. MoMask: Generative masked modeling of 3D human motions. In *CVPR*, 2024.

[9] S. Clavet. Motion matching and the road to next-gen animation. In *Game Developers Conference*, 2016.

[10] D. Holden, O. Kanoun, M. Perepichka, and T. Popa. Learned motion matching. *ACM Transactions on Graphics (SIGGRAPH)*, 2020.

[11] S. Starke, Y. Zhao, T. Komura, and K. Zaman. Local motion phases for learning multi-contact character movements. *ACM Transactions on Graphics (SIGGRAPH)*, 2020.

[12] H. Y. Ling, F. Zinno, G. Cheng, and M. van de Panne. Character controllers using motion VAEs. *ACM Transactions on Graphics (SIGGRAPH)*, 2020.

[13] F. G. Harvey, M. Yurick, D. Nowrouzezahrai, and C. Pal. Robust motion in-betweening. *ACM Transactions on Graphics (SIGGRAPH)*, 2020.

[14] S. Cohan, D. S. L. K. Reda, K. Wu, and M. van de Panne. Flexible motion in-betweening with diffusion models. In *SIGGRAPH*, 2024.

[15] Lucidrains. *adam-atan2*: Adam with the atan2 update rule. Software library.

文档状态：初稿（v0.1），2026-10-26。**所有数值取自仓库内已提交的证据文件，由** `paper/generate_figures.py` **直接读取并绘图，未在本文件中手工转录。**

作者：赵梓宁（独立研究者），联系 zzn1982638980@gmail.com。 **本报告不代表** NVIDIA **官方产品，也不表示获得** NVIDIA **的认可或背书。**